GPU-Accelerated Anisotropic Edge Detection via Orientation-Selective Stencil Filters

J. C. Vaught
Department of Mechanical Engineering
University of South Carolina
Columbia, SC 29208
jvaught@sc.edu

***Abstract*—We present three orientation-selective edge detection filters, the fused polynomial stencil, the rectangular Gaussian kernel, and the elliptical Gaussian kernel, unified under a common GPU execution framework. The filters compute image gradients by sweeping candidate edge orientations and selecting the direction of maximum normal-derivative response, incorporating hundreds of pixels into each estimate for inherent noise suppression without pre-smoothing. Our first contribution is a *fused stencil formulation* that algebraically collapses the multi-step Line Filter pipeline of Bagan and Wang into a single precomputed weight vector per orientation, achieving 24× speedup and 311× reduction in GPU memory consumption while producing identical output. Analysis of the fused weights reveals a structural limitation: approximately 28% of the effective kernel weight cancels to near-zero due to destructive interference between overlapping polynomial neighborhoods. This motivates our second contribution, two *geometric kernel alternatives* (rectangular and elliptical Gaussian envelopes) that define anisotropic derivative weights directly from spatial geometry, bypassing polynomial fitting entirely. The geometric kernels match the fused stencil's accuracy within 0.3% ODS F-score while running 3× faster, with provably uniform weight distributions. We evaluate all three variants across 1,200+ parameter configurations, three benchmarks (BSDS500, BIPED, UDED), 54 classical baselines, and five state-of-the-art deep learning models. On clean imagery, the proposed filters outperform every classical edge detector tested and approach deep learning accuracy without any training data. Under noise, the filters overtake all tested deep learning models below dataset-dependent signal-to-noise ratio thresholds through parametric adaptation. We place this work in the historical context of Savitzky–Golay filtering, steerable filters, and anisotropic Gaussian derivatives, connecting sixty years of local polynomial fitting literature to modern GPU-accelerated edge detection.**

***Index Terms*—Edge detection, Anisotropic filtering, GPU acceleration, Savitzky-Golay, Polynomial fitting, Noise robustness, Deep learning, Triton**

I. Introduction

Edge detection remains one of the most fundamental operations in computer vision. It underpins object recognition pipelines, drives contour-based image segmentation, enables autonomous navigation through structured environments, and provides the geometric primitives on which industrial inspection and dimensional measurement systems depend [1], [2]. Despite more than five decades of sustained research, the design of edge detection operators continues to be governed by a three-way tension between noise robustness, spatial precision, and computational cost. Increasing the spatial support of a gradient estimator improves its resilience to noise, but at the expense of blurring fine structural detail and increasing the number of arithmetic operations per pixel. No single operator has resolved this trade-off universally, and the history of the field can be read as a succession of attempts to shift the balance.

The earliest and still most widely deployed gradient operators are the small fixed-kernel convolutions introduced in the 1960s and 1970s. The Roberts Cross operator [3] estimates gradients along two diagonal directions using a 2×2 stencil. The Prewitt filter [4] and the Sobel filter [5] extend the support region to 3×3, incorporating a modest smoothing component orthogonal to the differentiation axis. These operators are trivially fast, require no tunable parameters, and are available in every major image-processing library. Their small spatial support, however, is also their principal limitation. With only nine pixels contributing to each gradient estimate, any noise within the window directly corrupts the output. Extending the kernel to 5×5 or 7×7 yields marginal improvement, but the hand-designed weight structures remain fixed and cannot adapt to the local edge orientation or the prevailing noise level in a given image region.

The Canny detector [1] addressed the noise sensitivity of small kernels by introducing a multi-stage pipeline. A Gaussian pre-smoothing step suppresses high-frequency noise before gradient computation, and subsequent non-maximum suppression and hysteresis thresholding stages refine the detected contours into thin, connected edge maps. The Canny detector became the dominant classical method and remains a standard baseline in the literature. Nevertheless, the Gaussian smoothing stage introduces a fundamental trade-off between denoising strength and localization accuracy. Stronger smoothing suppresses more noise but simultaneously displaces weak edges and low-contrast boundaries from their true positions. Furthermore, because the smoothing and differentiation stages are decoupled by design, the gradient orientation at each pixel is computed from an arctangent of two cardinal finite differences, propagating quantization and noise errors from the x and y channels into every intermediate angle estimate. These limitations are well documented but difficult to overcome within the Canny framework.

The past decade has seen a decisive shift toward deep learning approaches for boundary detection. Holistically-

Nested Edge Detection (HED) [6] demonstrated that a fully convolutional network with multi-scale side outputs could learn rich boundary representations from human-annotated ground truth. Subsequent architectures have pushed benchmark scores steadily higher. DexiNed [7] introduced dense extreme inception blocks that enable training from scratch without ImageNet pre-training. PiDiNet [8] integrated classical pixel-difference convolutions into a lightweight network architecture. TEED [9] achieved competitive accuracy with an extremely compact model of only 58K parameters. More recently, NBED [10] and DiffusionEdge [11] have explored negative-sample bootstrapping and diffusion-based generative modeling to further improve boundary recall. These methods routinely achieve Optimal Dataset Scale (ODS) F-scores above 0.80 on the BSDS500 benchmark [12], substantially outperforming all classical operators on that dataset. However, the reliance on supervised training introduces practical constraints that are difficult to ignore. Deep models require large corpora of pixel-level edge annotations, which are expensive to produce and inherently subjective [13]. More critically, models trained on natural photographic imagery generalize poorly when deployed in domains whose intensity statistics diverge significantly from the training distribution. Underwater imagery, medical scans, thermal infrared sequences, and synthetic-aperture radar all present noise characteristics that are poorly represented in standard benchmarks [14], [15]. Re-training or fine-tuning for each new domain negates the convenience that motivates a general-purpose detector. Finally, learned operators produce edge probability maps rather than calibrated derivative estimates, making them unsuitable for applications that require quantitative gradient magnitudes or precise orientation angles.

Each generation of methods, then, trades one problem for another. Classical fixed-kernel operators are fast and deterministic but fragile under noise. Multi-stage pipelines like Canny provide better noise handling but blur weak edges and decouple smoothing from differentiation. Deep learning models achieve the highest benchmark scores but depend on training data, generalize poorly across domains, and abandon the notion of a true image derivative.

A. The Local Polynomial Fitting Approach

An alternative strategy, one that couples smoothing and differentiation into a single principled operation, has been independently reinvented multiple times across disconnected communities over the past six decades. The idea is simple. Rather than convolving the image with a fixed kernel designed by hand, one fits a low-order polynomial to the pixel intensities in a local neighborhood and reads off the desired derivative from the fitted coefficients. Because the polynomial fit is a least-squares operation over a potentially large number of pixels, it provides inherent noise suppression whose strength scales with the neighborhood size. Because the derivative is extracted analytically from the polynomial model, smoothing and differentiation are fused into one step.

Savitzky and Golay [16] introduced this approach in 1964 for smoothing and differentiating spectroscopic data. Their key insight was that the least-squares polynomial fit over an equally spaced one-dimensional window can be expressed as a single convolution with precomputed weights, making it no more expensive than a moving average. The method became ubiquitous in analytical chemistry and signal processing. Gorry [17] generalized the convolution coefficients to arbitrary derivative orders and non-uniform spacing. Extensions to two-dimensional grids followed. Meer, Baugher, and Rosenfeld [18] analyzed the frequency-domain properties of 2D polynomial fitting kernels for image pyramid generation. Luo *et al.* [19] provided a systematic treatment of two-dimensional Savitzky–Golay digital differentiators, deriving their transfer functions and establishing conditions on the polynomial order and window size needed for accurate derivative estimation on discrete image grids.

In a parallel line of development, Freeman and Adelson [20] solved the problem of orientation-selective filtering analytically by showing that certain filter families can be *steered* to any arbitrary angle through a linear combination of a small set of basis filters. Steerable filters provide a computationally elegant means of computing directional derivatives without the brute-force cost of evaluating a separate kernel at each candidate orientation. Gaussian derivatives are the canonical example. A first-order Gaussian derivative steered to angle θ is simply the cosine-weighted sum of the x- and y-derivative basis filters, and the maximum-response orientation can be found in closed form. This framework has been widely adopted in texture analysis, motion estimation, and feature detection.

Geusebroek, Smeulders, and van de Weijer [21] built on this foundation by constructing *anisotropic* Gaussian derivative kernels in which the smoothing envelope is elongated along the edge direction. By decoupling the scale parameters along the normal and tangent axes, their filters can integrate information over a long tangential extent for noise suppression while maintaining a narrow profile in the normal direction for precise localization. The resulting kernels are separable along the principal axes of the elliptical Gaussian, enabling efficient recursive implementation.

Bagan and Wang [22], [23] independently arrived at what is, in algebraic terms, a two-dimensional Savitzky–Golay derivative filter applied in a vision context. Their initial formulation, the Weighted Vector Filter (WVF) [22], fits a two-dimensional Taylor polynomial of degree d to pixels gathered from a circular neighborhood and extracts the gradient from the fitted coefficients, precisely the 2D Savitzky–Golay procedure. Their subsequent extension, the Line Filter (LF) [23], evaluates the point-filter response at multiple virtual positions displaced along each candidate edge direction, effectively constructing an anisotropic support region that is elongated tangentially to the hypothesized boundary. A brute-force sweep over N_s uniformly spaced orientations selects the direction producing the strongest normal derivative. This orientation sweep is a discrete, sampled-angle approximation to the continuous steerability property formalized by Freeman and Adelson three decades earlier. The publications introducing the WVF and LF do not reference the Savitzky–Golay literature, steerable filters, or anisotropic Gaussian derivatives.

The connection between these bodies of work appears to have gone entirely unrecognized.

The consequence of this fragmentation is that a natural and productive question has never been posed. If the Line Filter is, at its core, an oriented, line-averaged two-dimensional Savitzky–Golay derivative filter, what is the simplest and fastest formulation that preserves its favorable accuracy characteristics? Conversely, what structural limitations does the polynomial-fitting construction impose, and can alternative kernel geometries avoid those limitations while retaining the same noise-robustness properties?

B. Questions and Contributions

This paper addresses three specific questions. First, can the multi-step LF pipeline, in which a point filter is repeatedly evaluated at shifted positions along each candidate orientation and the responses are aggregated, be collapsed into a single linear operation suitable for GPU execution? Second, what structural limitations does this algebraic fusion reveal about polynomial-fitting-based kernels? Third, can simpler geometric kernel definitions, such as rectangular and elliptical Gaussian envelopes, match or exceed the accuracy of polynomial fitting while avoiding those limitations?

The contributions are as follows.

1) *Fused stencil formulation.* We show that the entire LF pipeline can be algebraically collapsed into a single precomputed weight vector per orientation, reducing the filter to one gather-multiply-sum operation per pixel. This reformulation eliminates all redundant memory accesses and yields a 24 × wall-clock speedup and a 311 × reduction in GPU VRAM consumption compared to a naive implementation, while preserving bit-identical output.
2) *Weight cancellation analysis.* The stencil fusion exposes a structural deficiency that is invisible in the unfused pipeline. Because neighboring point-filter evaluations share large fractions of their pixel neighborhoods, many pixels receive contributions from multiple overlapping fits with opposing signs. We demonstrate that approximately 28% of the total weight budget in a typical fused stencil is consumed by pixels whose net weight magnitude is negligible, representing wasted computation and a suboptimal allocation of the filter's effective degrees of freedom.
3) *Geometric kernel alternatives.* We propose two kernel families that define the anisotropic weight envelope directly from an analytic function, bypassing the tile-and-sum construction entirely. A rectangular kernel assigns uniform weight within an oriented bounding box, while an elliptical Gaussian kernel produces a smooth, naturally tapered distribution whose major axis is aligned with the edge tangent. Both alternatives eliminate weight cancellation by construction, match polynomial fitting accuracy to within 0.3% ODS F-score, and execute approximately 3 × faster than the fused polynomial stencil.
4) *Comprehensive evaluation.* We evaluate all filter variants across more than 1,200 parameter configurations on three standard benchmarks (BSDS500, BIPED, and UDED), comparing against 54 classical baseline operators and 5 state-of-the-art deep learning models. The evaluation includes noise robustness testing under additive Gaussian corruption at multiple severity levels, providing the most extensive comparative assessment of polynomial-fitting-based edge detectors to date.

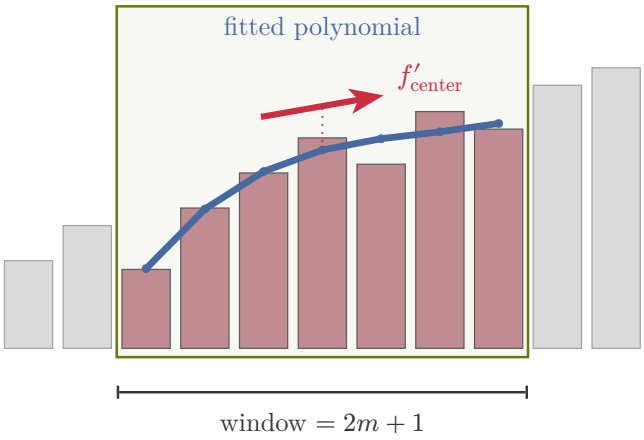


Fig. 1. One-dimensional Savitzky–Golay filtering. A polynomial of degree d is fitted to a sliding window of $2m + 1$ samples (highlighted), and the derivative at the center point is extracted from the fitted coefficients. This example uses degree $d = 3$ and half-width $m = 3$ (window size 7).

II. Related Work

The filters examined in this paper, the Wide View Filter (WVF) [22] and the Line Filter (LF) [23], are presented by their authors as novel contributions to edge detection. In this section we situate them within a sixty-year lineage of local polynomial fitting, orientation-selective filtering, and classical gradient estimation. The central observation is that the WVF and LF do not introduce new mathematical machinery. Rather, they assemble well-known components, specifically least-squares polynomial fitting over shaped support regions with orientation sweeping, into a particular configuration. Understanding these antecedents is essential for a fair evaluation of what, if anything, the WVF and LF add beyond existing methods.

A. Local Polynomial Fitting for Derivative Estimation

The idea of fitting a low-degree polynomial to data within a sliding window and then differentiating the polynomial to estimate derivatives dates to the foundational work of Savitzky and Golay [16]. Their 1964 paper showed that the convolution weights for smoothing and differentiation can be precomputed from the pseudoinverse of a Vandermonde matrix constructed over the sample indices. For a one-dimensional window of $2m + 1$ samples and a polynomial of degree p, the k-th derivative weights are the k-th row of $\left(\boldsymbol{V}^\top \boldsymbol{V}\right)^{-1} \boldsymbol{V}^\top$, where $\boldsymbol{V}$ is the Vandermonde matrix. This formulation yields finite impulse response (FIR) filters whose coefficients depend only on the window size and polynomial degree, not on the data itself. The resulting filters are optimal in the least-squares sense and provide simultaneous smoothing and differentiation.

Gorry [17] generalized the Savitzky–Golay framework by deriving recursive relations for the convolution coefficients that extend naturally to higher-order derivatives and non-uniform sample spacing. Extensions to two-dimensional domains followed. Meer et al. [18] analyzed the frequency-domain properties of 2D polynomial fitting kernels in the context of image pyramids, establishing connections between polynomial order, window shape, and spectral characteristics. Luo et al. [19] provided a systematic analysis of the properties of Savitzky–Golay digital differentiators, including their frequency response and noise attenuation characteristics, and gave explicit constructions for 2D variants.

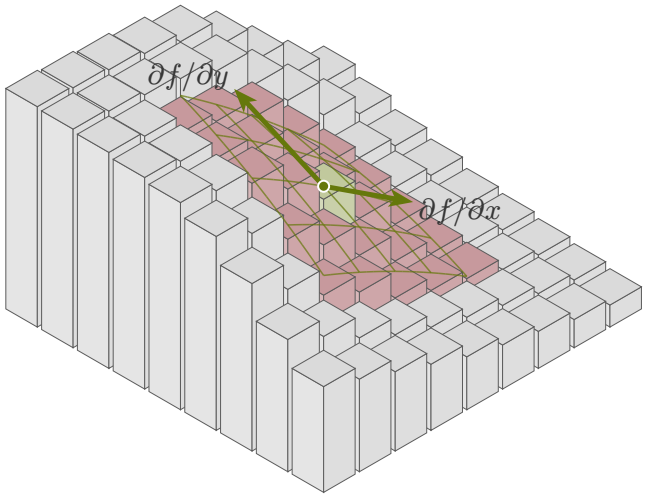


Fig. 2. Two-dimensional Savitzky–Golay filtering on a 9×9 pixel grid. Column heights represent pixel intensities. The inner 5×5 window (garnet) is fitted with a degree $d = 2$ polynomial surface (green wireframe), and the partial derivatives $\partial f/\partial x$ and $\partial f/\partial y$ are extracted at the center pixel (green dot). Gray columns outside the window are not used in the fit.

A closely related thread is the locally weighted scatterplot smoothing (LOWESS) framework of Cleveland [24], which fits weighted polynomials within a neighborhood where the weights decay with distance from the center point. LOWESS differs from the standard Savitzky–Golay approach primarily in using a smooth distance-based weighting function rather than a uniform or binary window. The practical effect is that LOWESS produces derivative estimates that are influenced more strongly by nearby samples, at the cost of losing the precomputed-weight efficiency of Savitzky–Golay.

With this context, the WVF [22] can be understood as a 2D Savitzky–Golay filter with three specific design choices. First, the support region is circular (a disk of radius r) rather than the rectangular windows typical of the classical formulation. Second, the coordinate system is rotated so that one polynomial axis aligns with a candidate edge orientation θ_k. Third, the filter is evaluated at K discrete orientations and the orientation yielding the maximum gradient response is selected.

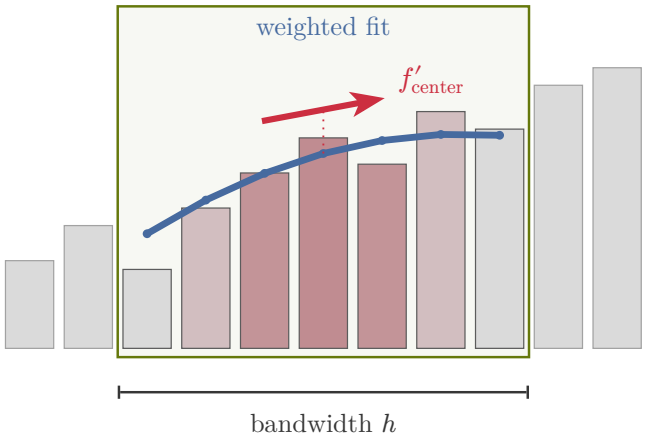


Fig. 3. LOWESS (Cleveland, 1979). Samples within the bandwidth are weighted by a distance-based kernel (tricube) from the target location; bar color encodes weight (darker garnet = higher weight, gray = low weight). This example uses bandwidth $h = 3$ and polynomial degree $d = 2$.

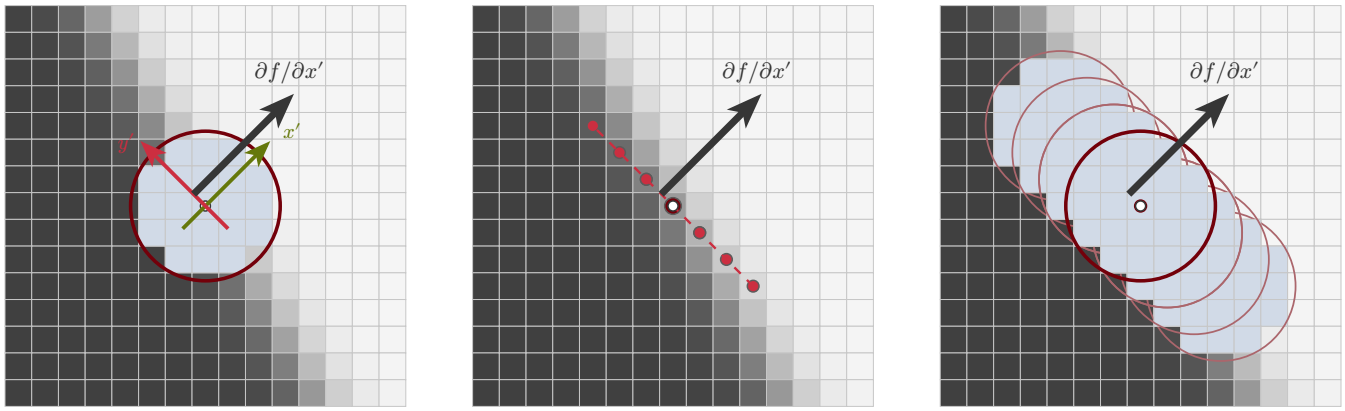
Each of these choices, circular support, coordinate rotation, and orientation sweeping, has independent precedent in the literature. The LF [23] extends the WVF by averaging polynomial derivative estimates computed at *virtual expansion points* distributed along a line perpendicular to the candidate edge direction, effectively replacing the single disk-shaped support with a rectangular strip sampled at multiple centers. This construction is a specific instance of line-averaged local polynomial regression, a technique that appears in the signal processing literature under various names.

B. Orientation-Selective Filtering

The problem of estimating image derivatives along arbitrary orientations has been addressed extensively through steerable filters. Freeman and Adelson [20] demonstrated that certain classes of filters, including Gaussian derivatives up to arbitrary order, can be analytically *steered* to any orientation by taking a linear combination of a small, fixed set of basis filters. For a filter of angular order n, only $n + 1$ basis responses are required. The WVF's strategy of computing filter responses at K discrete orientations and selecting the maximum is a brute-force discretization of the same underlying operation. Where steerable filters achieve continuous orientation interpolation via analytic basis decomposition, the WVF evaluates responses on a discrete grid, trading elegance and computational efficiency for implementation simplicity.

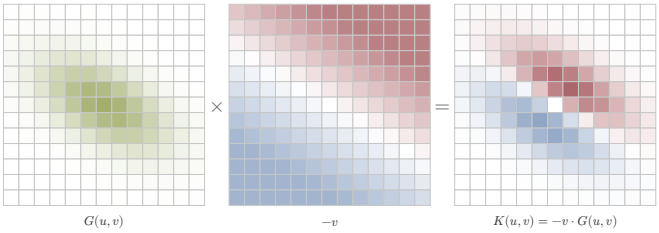
Phase-based methods provide an alternative route to orientation estimation. Morrone and Owens [25] proposed detecting features at points of maximum *local energy*, defined through the analytic signal and its phase. Perona [26] developed steerable and scalable kernels that combine orientation selectivity with scale adaptation, using the phase congruency of oriented filter responses to localize edges independently of contrast. These phase-based approaches offer a principled framework for orientation estimation that does not require the exhaustive search employed by the WVF.

Anisotropic Gaussian derivatives represent yet another strategy. Geusebroek et al. [21] developed efficient algorithms



(a) Wide View Filter (b) Virt. Pixels (c) Line Filter

Fig. 4. The WVF and LF as oriented 2D Savitzky–Golay filters. (a) The Wide View Filter selects a circular neighborhood of N_p pixels around the target pixel, rotates local coordinates (x', y') to align with a candidate edge orientation, and extracts the normal derivative $\partial f/\partial x'$ via polynomial fitting. (b) The Line Filter places $(2m+1)$ virtual evaluation points along the tangent direction y', each centered on the nearest pixel. (c) At each virtual point, a full circular neighborhood is gathered and a polynomial fit applied. The derivative estimates are combined via Gaussian-weighted averaging. This example uses $m = 4$ (9 virtual points).



(a) Gaussian Envelope (b) Derivative Profile (c) Combined Kernel

Fig. 5. Anisotropic Gaussian derivatives (Geusebroek et al., 2003). An elongated Gaussian envelope aligned along the candidate edge direction is modulated by the normal derivative profile $(-v)$, producing the dipole pattern that is essentially identical to the geometric kernels proposed in this work.

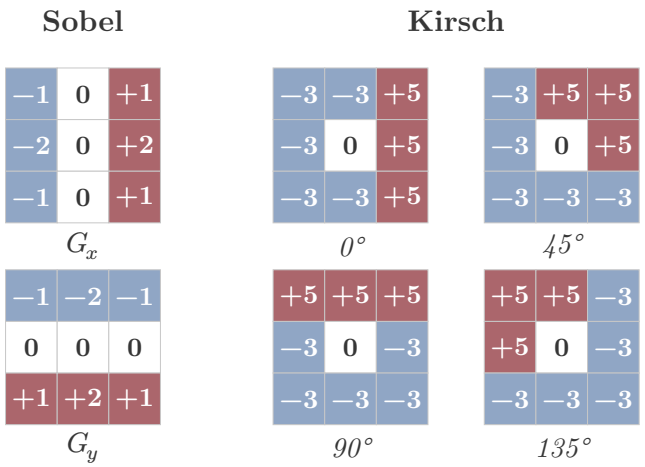


Fig. 6. Classical 3×3 edge operators. Left column: Sobel (1968) estimates horizontal and vertical gradients with two fixed kernels G_x and G_y. Center and right columns: Kirsch (1971) evaluates eight compass masks (four shown) and selects the maximum response. Positive weights in garnet, negative in blue, center pixel in white.

for computing derivatives of anisotropic (elongated) Gaussian kernels, enabling the construction of filters whose spatial support adapts to local image structure. The geometric relationship between their elongated Gaussian kernels and the oriented disk or strip supports of the WVF and LF is direct. Both families of methods implement the same core idea: an anisotropic, orientation-dependent smoothing kernel whose derivative provides a directional gradient estimate. The principal difference is that the Gaussian formulation admits closed-form expressions and separable implementations, while the WVF and LF rely on numerical construction of their weight matrices.

C. Classical Edge Detection

The earliest edge detection operators were small, hand-designed convolution masks. Roberts [3] introduced 2×2 cross-difference operators for detecting diagonal edges. Sobel [5] and Prewitt [4] independently proposed 3×3 masks that approximate first-order directional derivatives with a degree of smoothing orthogonal to the gradient direction. Kirsch [27] designed a set of eight 3×3 templates, one for each compass direction, and defined the edge response as the maximum over all eight. The Frei–Chen basis [28] decomposed the 3×3 neighborhood into nine orthogonal subspaces, separating edge, line, and average components, and detected edges by projection onto the edge subspace. All of these operators share the property of fixed, small spatial support, which limits their noise robustness and their ability to adapt to edges at different scales.

The Gaussian derivative framework placed edge detection on a firmer mathematical foundation. Marr and Hildreth [29] proposed detecting zero crossings of the Laplacian of Gaussian ($\nabla^2 G$), providing a principled coupling of smoothing scale and differentiation. Canny [1] formulated edge detection as an optimization problem, seeking the operator that maximizes detection probability and localization accuracy while minimizing multiple responses. His solution for step edges in white Gaussian noise is closely approximated by the first

(a) Input (b) Smoothed (c) Gradient (d) Direction (e) Edges

Fig. 7. The Canny edge detection pipeline demonstrated on a real image. (a) Grayscale input. (b) Gaussian smoothing ($\sigma = 2.0$) reduces noise but blurs fine edges. (c) Gradient magnitude computed via Sobel on the smoothed image. (d) Gradient direction encoded as hue (brightness proportional to magnitude). (e) Non-maximum suppression and hysteresis thresholding produce thin, connected edge contours.

derivative of a Gaussian, and his framework introduced non-maximum suppression and hysteresis thresholding as post-processing stages that remain standard practice. Lindeberg [30] extended these ideas to *automatic scale selection*, using normalized derivatives across scale space to identify the characteristic scale of each edge. A persistent limitation of all fixed-kernel methods, whether 3×3 masks or Gaussian derivatives at a single scale, is that their spatial support does not adapt to the local noise level or to the spatial extent of the edge structure. The WVF and LF address this limitation by enlarging the support region, but as discussed in Section II.A, the mechanism they use, polynomial fitting over large windows, is itself classical.

D. Deep Learning Edge Detection

The introduction of deep convolutional networks to edge detection marked a paradigm shift in performance on standard benchmarks. Holistically-Nested Edge Detection (HED) [6] pioneered the use of multi-scale side outputs from a VGG-style backbone, fusing predictions from multiple network depths to produce edge maps that capture both fine details and large-scale contours. Structured Forests [13], while not a deep method, introduced the random-forest-based approach to structured prediction for edges that motivated much subsequent work. DexiNed [7] extended the multi-scale fusion paradigm with a dense extreme inception architecture, demonstrating strong generalization to datasets unseen during training. The Tiny and Efficient Edge Detector (TEED) [9] achieved competitive accuracy with a dramatically reduced parameter count, demonstrating that architectural efficiency and edge detection quality need not be in tension. PIDINet [8] incorporated traditional pixel-difference operations as inductive biases within a lightweight network, explicitly bridging classical gradient computation and learned feature extraction. More recently, NBED [10] proposed a neural-network-based architecture with attention mechanisms for boundary detection, and DiffusionEdge [11] applied diffusion probabilistic models to generate crisp edge maps through iterative denoising.

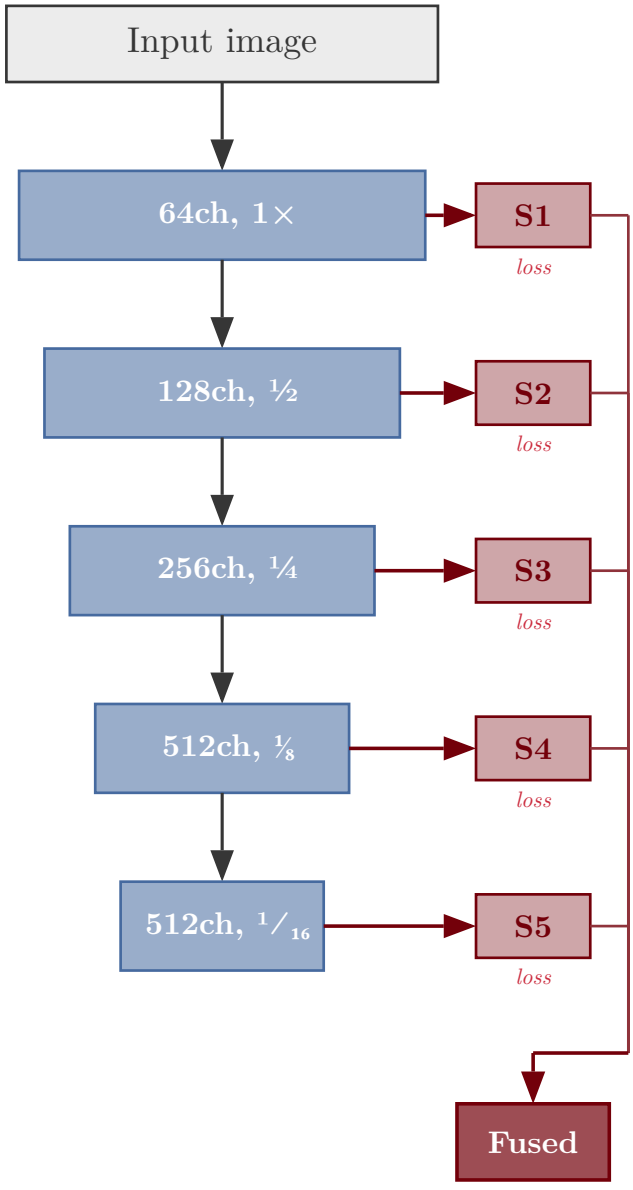


Fig. 8. Holistically-Nested Edge Detection (Xie and Tu, 2015). A VGG-style encoder produces multi-scale side outputs S_1 through S_5 that are fused into a final edge map. Each side output captures edges at a different spatial scale.

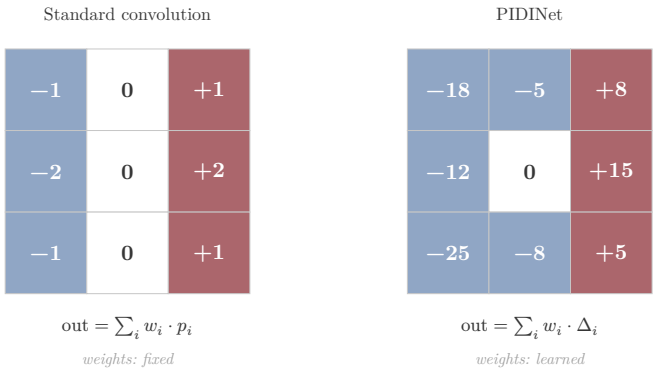


Fig. 9. PIDINet pixel difference convolutions (Su et al., 2021). Instead of multiplying pixel values by learned weights, PIDINet computes pixel *differences* Δ_i between the center and each neighbor, then applies learned weights w_i to the differences. This is structurally similar to a classical gradient operator but with trainable coefficients.

These methods achieve state-of-the-art performance on benchmarks such as BSDS500 [12], but they share several limitations that motivate continued interest in classical approaches. First, they depend on labeled training data, and their performance degrades under *domain shift* when the test distribution differs from the training distribution. Underwater imagery, medical imaging, and satellite data all present domains where labeled edge data is scarce. Second, the learned filters are opaque. One cannot, in general, characterize the spatial frequency response, noise sensitivity, or derivative accuracy of a trained network's edge output in the way that one can for a Savitzky–Golay filter or a Gaussian derivative. Third, deep methods provide no formal guarantees on gradient accuracy. The WVF and LF, by virtue of their polynomial fitting formulation, admit closed-form bias and variance expressions that can guide parameter selection, an advantage that no data-driven method currently shares.

E. GPU Acceleration for Image Filtering

The computational cost of applying large-support filters at multiple orientations has historically been a barrier to deploying methods like the WVF and LF in real-time applications. Modern GPU computing frameworks have substantially lowered this barrier. The cuDNN library [31] provides highly optimized implementations of standard convolution operations, including batched and grouped convolutions that can be leveraged for multi-orientation filtering. For operations that do not map cleanly onto standard convolution, the Triton compiler [32] enables rapid development of custom GPU kernels in a high-level Python-embedded language, with automatic tiling and memory management. CUTLASS [33] provides composable CUDA templates for general matrix operations that can express the gather-and-dot-product structure underlying all three filter variants considered in this work.

The key observation for GPU implementation is that the WVF, LF, and the anisotropic filter variant proposed in this paper all reduce to the same computational primitive. For each pixel and each candidate orientation, a set of pixel values is gathered from the support region and multiplied by a precomputed weight vector. This gather-dot-product operation is embarrassingly parallel across pixels and orientations, making it well-suited to GPU execution. The orientation sweep, which the WVF performs sequentially, can be issued as a batched operation over the K orientation channels, and the subsequent maximum selection is a trivial reduction. The practical consequence is that even the WVF's brute-force orientation search becomes tractable when implemented as a single fused GPU kernel, removing much of the computational motivation for the analytic interpolation provided by steerable filters.

III. Mathematical Framework

This section develops the mathematical framework underlying the Anisotropic Savitzky–Golay Filter (ASF). The central idea is to estimate image gradients by fitting a local polynomial surface at each pixel, with the coordinate system rotated to align with each candidate edge orientation. We begin with the polynomial model, derive the least-squares estimator, describe the orientation sweep, define the neighbor selection strategy, and introduce the line extension that provides anisotropic noise averaging along the edge tangent.

A. Local Polynomial Model

Consider a grayscale image $f : \mathbb{Z}^2 \to \mathbb{R}$ and a target pixel at global coordinates (X_0, Y_0). The goal is to estimate the gradient magnitude and orientation at this pixel by fitting a two-dimensional polynomial to its local neighborhood. The key distinction from conventional gradient operators is that the fitting coordinate system is rotated to align with each candidate edge orientation, so that the normal derivative (perpendicular to the putative edge) can be read off directly from the polynomial coefficients.

For a candidate orientation θ_k, a local coordinate system (x, y) is defined with x along the edge normal and y along the edge tangent. The transformation from global to local coordinates for any neighbor pixel (X_i, Y_i) is given by the standard rotation

$$x_i = (X_i - X_0)\cos\theta_k + (Y_i - Y_0)\sin\theta_k \tag{1}$$

$$y_i = -(X_i - X_0)\sin\theta_k + (Y_i - Y_0)\cos\theta_k \tag{2}$$

which can be written compactly in matrix form as

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} \cos\theta_k & \sin\theta_k \\ -\sin\theta_k & \cos\theta_k \end{pmatrix} \begin{pmatrix} X_i - X_0 \\ Y_i - Y_0 \end{pmatrix} \tag{3}$$

The intensity surface in the neighborhood of the origin is then approximated by a two-dimensional Taylor expansion of order d. For $d = 2$, this takes the explicit form

$$f(x_i, y_i) \approx c_0 + c_1 x_i + c_2 y_i + c_3 \frac{x_i^2}{2} + c_4 \frac{y_i^2}{2} + c_5 x_i y_i \tag{4}$$

where $c_0 = f^0$ is the intensity at the origin, $c_1 = f^0_x$ and $c_2 = f^0_y$ are the first partial derivatives, and $c_3 = f^0_{xx}$, $c_4 = f^0_{yy}$, $c_5 = f^0_{xy}$ are the second partials. The general expansion for arbitrary order d is

$$f(x_i, y_i) \approx \sum_{p+q \le d} \frac{f^0_{\underbrace{x\cdots x}_{p}\,\underbrace{y\cdots y}_{q}}}{p! \cdot q!} \cdot x_i^p y_i^q \tag{5}$$

The number of unknown coefficients in this expansion is

$$M = \frac{(d+1)(d+2)}{2} \tag{6}$$

For common polynomial orders, M takes the values 3 ($d = 1$), 6 ($d = 2$), 10 ($d = 3$), and 15 ($d = 4$). Collecting these unknowns into a coefficient vector $\boldsymbol{z} \in \mathbb{R}^M$, the approximation at pixel i becomes the inner product of $\boldsymbol{z}$ with a monomial basis vector evaluated at (x_i, y_i).

This local polynomial model has a direct connection to the classical Savitzky–Golay filter [16]. In the one-dimensional case, fitting a polynomial of degree d to a uniformly-spaced window and evaluating the n-th derivative at the center is precisely the Savitzky–Golay procedure. The present formulation generalizes this to two dimensions, non-uniform (integer-grid) sampling, and orientation-dependent coordinate systems. The "Savitzky–Golay" designation in the filter name reflects this lineage.

B. Least-Squares Gradient Estimation

Given N_p neighbor pixels surrounding (X_0, Y_0), the Taylor expansion (5) evaluated at each neighbor yields an overdetermined linear system. Define the design matrix $\boldsymbol{A}_{\theta_k} \in \mathbb{R}^{N_p \times M}$ whose i-th row contains the monomial basis evaluated at the local coordinates (x_i, y_i) of the i-th neighbor. For polynomial order $d = 2$, row i of the design matrix is

$$\boldsymbol{A}_{\theta_k}[i,:] = \begin{pmatrix} 1 \\ x_i \\ y_i \\ \frac{x_i^2}{2} \\ \frac{y_i^2}{2} \\ x_i y_i \end{pmatrix}^\top \tag{7}$$

The system to be solved is then

$$\boldsymbol{A}_{\theta_k} \boldsymbol{z} = \boldsymbol{b} \tag{8}$$

where $\boldsymbol{b} = \big(f(X_1, Y_1), ..., f\big(X_{N_p}, Y_{N_p}\big)\big)^\top \in \mathbb{R}^{N_p}$ is the vector of observed pixel intensities at the neighbor locations. Because $N_p > M$, this system is overdetermined and generally has no exact solution.

The ordinary least-squares (OLS) solution minimizes the sum of squared residuals and is given by the normal equations

$$\hat{\boldsymbol{z}} = \left(\boldsymbol{A}_{\theta_k}^\top \boldsymbol{A}_{\theta_k}\right)^{-1} \boldsymbol{A}_{\theta_k}^\top \boldsymbol{b} \tag{9}$$

This can be written compactly using the Moore–Penrose pseudoinverse $\boldsymbol{P}_{\theta_k} = \boldsymbol{A}_{\theta_k}^+$ as

$$\hat{\boldsymbol{z}} = \boldsymbol{P}_{\theta_k} \boldsymbol{b}, \quad \boldsymbol{P}_{\theta_k} = \left(\boldsymbol{A}_{\theta_k}^\top \boldsymbol{A}_{\theta_k}\right)^{-1} \boldsymbol{A}_{\theta_k}^\top \in \mathbb{R}^{M \times N_p} \tag{10}$$

A critical observation is that the pseudoinverse $\boldsymbol{P}_{\theta_k}$ depends only on the geometry of the neighborhood (the relative pixel positions and the orientation angle θ_k), not on the pixel intensities. Since the neighborhood geometry is the same for every pixel in the image (assuming a fixed N_p and θ_k), the pseudoinverse can be *precomputed* once for each orientation and reused across the entire image. This converts what would otherwise be a per-pixel matrix inversion into a single dot product.

The estimated normal derivative, which is the quantity of interest for edge detection, is the coefficient $\hat{z}_1 = \hat{f}_x$. This corresponds to row 1 (zero-indexed) of the pseudoinverse matrix, yielding

$$\hat{f}_x^{(k)} = \boldsymbol{p}_{\text{fx}}^{(k)} \cdot \boldsymbol{b}, \quad \boldsymbol{p}_{\text{fx}}^{(k)} = \boldsymbol{P}_{\theta_k}[1,:] \in \mathbb{R}^{N_p} \tag{11}$$

The gradient estimate at orientation θ_k thus reduces to a single inner product of the precomputed weight vector $\boldsymbol{p}_{\text{fx}}^{(k)}$ with the local pixel intensities $\boldsymbol{b}$. This structure is shared with all linear filter operations, but the key distinction is that the weights $\boldsymbol{p}_{\text{fx}}^{(k)}$ are derived from the polynomial fitting framework rather than from a handcrafted kernel.

C. Orientation Sweep and Maximum-Response Selection

Classical gradient-based edge detectors such as the Sobel operator and the Canny detector [1] estimate the gradient by computing partial derivatives along two fixed axes (x and y) and then combining them. The gradient magnitude is obtained as $G = \sqrt{G_x^2 + G_y^2}$ and the orientation as $\theta = \arctan\big(G_y, G_x\big)$. This two-direction approach implicitly assumes that the gradient can be accurately decomposed into two orthogonal components, which is reasonable for smooth intensity fields but introduces angular quantization error when the edge orientation does not align with one of the two axes.

The ASF takes a fundamentally different approach. Rather than fixing two filter directions and computing the gradient as a derived quantity, it evaluates the normal derivative *directly* at each of N_s candidate orientations uniformly distributed over the half-circle

$$\theta_k = \frac{k \cdot \pi}{N_s}, \quad k = 0, 1, ..., N_s - 1 \tag{12}$$

At each orientation, the polynomial fitting procedure of Section III.B produces a normal derivative estimate $\hat{f}_x^{(k)}$ via (11). The filter response and estimated orientation are then selected by the maximum-response criterion

$$k^* = \arg\max_k |\hat{f}_x^{(k)}| \tag{13}$$

$$G = |\hat{f}_x^{(k^*)}| \tag{14}$$

$$\theta^* = \theta_{k^*} \tag{15}$$

This sweep-and-select strategy has several advantages. First, it avoids the angular bias of two-direction schemes because the polynomial fit is optimally aligned with each candidate orientation. At the true edge direction, the x-axis of the local coordinate system is perpendicular to the edge, and the Taylor expansion captures the intensity transition most accurately. Second, the angular resolution of the orientation estimate is $\frac{\pi}{N_s}$ and can be made arbitrarily fine by increasing N_s, at a linear cost in computation. Third, the maximum response G is always at least as large as the response that would be obtained at any fixed pair of directions, making the ASF inherently more sensitive to weak edges whose orientations fall between the cardinal axes.

The half-circle range $[0, \pi)$ suffices because the normal derivative changes sign (but not magnitude) under a π rotation. The unsigned magnitude $|\hat{f}_x^{(k)}|$ is invariant to this sign flip, so orientations θ_k and $\theta_k + \pi$ produce identical responses.

D. Neighbor Selection

The choice of the N_p neighbor pixels determines the spatial support of the filter. The ASF selects the N_p integer-coordinate pixels closest to the target pixel (X_0, Y_0) in Euclidean distance, yielding an approximately circular support region. The radius of this region scales as

$$r \approx \left\lceil \sqrt{\frac{N_p}{\pi}} \right\rceil + 1 \tag{16}$$

For example, $N_p = 100$ produces a circular patch of radius approximately 7 pixels. The support is constant across all orientations; only the coordinate system rotates.

The *overdetermination ratio* $\frac{N_p}{M}$ governs the balance between noise suppression and spatial fidelity. A larger ratio provides more averaging and hence stronger noise reduction, at the cost of potentially smoothing fine spatial detail. For the default parameters $N_p = 100$ and $d = 4$ (giving $M = 15$), the ratio is

$$\frac{N_p}{M} = \frac{100}{15} \approx 6.7 \tag{17}$$

This means the system has approximately 6.7 times more equations than unknowns, providing substantial noise averaging while retaining the polynomial's ability to represent intensity variations up to fourth order.

An important distinction from Gaussian smoothing is that the polynomial fit does not blur edges isotropically. A Gaussian kernel attenuates all spatial frequencies uniformly within its support, including the step-like intensity transition that defines an edge. The polynomial model, by contrast, is *capable* of representing such transitions. A step edge along the y-axis (tangent direction) manifests as a large $c_1 = f_x^0$ coefficient, which the least-squares fit preserves rather than suppresses. The smoothing occurs only in the sense that the polynomial cannot represent intensity variations of order higher than d within the support. This property is fundamental to why the ASF achieves simultaneous noise suppression and edge preservation.

E. Line Extension

The polynomial fit described above operates on a compact, approximately circular neighborhood centered on a single pixel. While this point-filter formulation is effective for strong edges, it may produce unreliable responses at low-contrast boundaries where the intensity transition is too weak relative to the noise level. The line extension addresses this limitation by extending the support region anisotropically along the candidate edge direction.

For a given orientation θ_k, $(2m+1)$ virtual evaluation positions are placed along the edge tangent direction at unit-pixel spacing

$$(X_j, Y_j) = \left(X_0 + j\cos\left(\theta_k + \frac{\pi}{2}\right), Y_0 + j\sin\left(\theta_k + \frac{\pi}{2}\right)\right), \quad j \in \{-m, \ldots, 0, \ldots, m\} \tag{18}$$

where the offset direction $\theta_k + \frac{\pi}{2}$ corresponds to the edge tangent (perpendicular to the normal direction θ_k). At each position j, the full polynomial fitting procedure is applied independently. The N_p nearest neighbors of (X_j, Y_j) are gathered, the design matrix is constructed at orientation θ_k, and the normal derivative $\hat{f}_x^{(j,k)}$ is extracted via the pseudoinverse weight vector. These per-position derivative estimates are then combined via Gaussian-weighted averaging

$$R_k = \sum_{j=-m}^{m} w_j \cdot \hat{f}_x^{(j,k)} \tag{19}$$

where the weights follow a Gaussian profile

$$w_j = \exp\left(-\frac{j^2}{2\sigma_\ell^2}\right), \quad \sigma_\ell = \frac{m}{2} \tag{20}$$

The maximum-response selection is applied to the line-extended responses rather than to the point-filter responses

$$k^* = \arg\max_k |R_k|, \quad G = |R_{k^*}| \tag{21}$$

The Gaussian weighting ensures that positions near the center of the line segment contribute more than those at the periphery, providing a smooth taper that avoids boundary artifacts. The parameter m controls the half-length of the line extension. For $m = 7$, the line segment spans 15 pixels along the edge tangent, giving the filter access to a substantially larger spatial context than the point filter alone.

The line extension provides two complementary benefits. First, it averages derivative estimates from multiple positions along the edge, reducing the variance of the gradient magnitude estimate by a factor proportional to the effective number of independent samples. Second, it enforces spatial coherence along the edge tangent. If a true edge runs through the target pixel, the derivative estimates at neighboring positions along the tangent direction will be consistently large, reinforcing the response. Conversely, isolated noise spikes, which are spatially uncorrelated, will not produce coherent responses along the line and will therefore be suppressed.

The computational cost of the line extension is significant. Each orientation requires $(2m+1)$ independent polynomial fits, and each fit gathers N_p pixel intensities from the image. The total number of memory accesses per pixel per orientation is therefore $(2m+1) \times N_p$. For $m = 7$ and $N_p = 100$, this amounts to 1500 gathers per orientation, and with $N_s = 36$ orientations the total reaches 54000 gathers per pixel. Many of these accesses are redundant, as neighboring evaluation positions share overlapping neighborhoods. This redundancy motivates the fused stencil formulation developed in the following section, which algebraically collapses the multi-step pipeline into a single weighted gather per pixel per orientation.

IV. Fused Stencil Formulation

The line-extended anisotropic steerable filter described in the previous section computes the orientation response R_k through a sequence of $(2m+1)$ independent polynomial fits, each requiring a separate gather of N_p pixel intensities, coordinate rotation, and pseudoinverse multiplication. For practical line-extension lengths ($m = 7$ implies 15 polynomial fits per orientation), this cascade dominates the computational cost and exhibits significant redundancy, since adjacent fit positions share most of their circular neighborhoods. This section derives an algebraic reformulation that collapses the

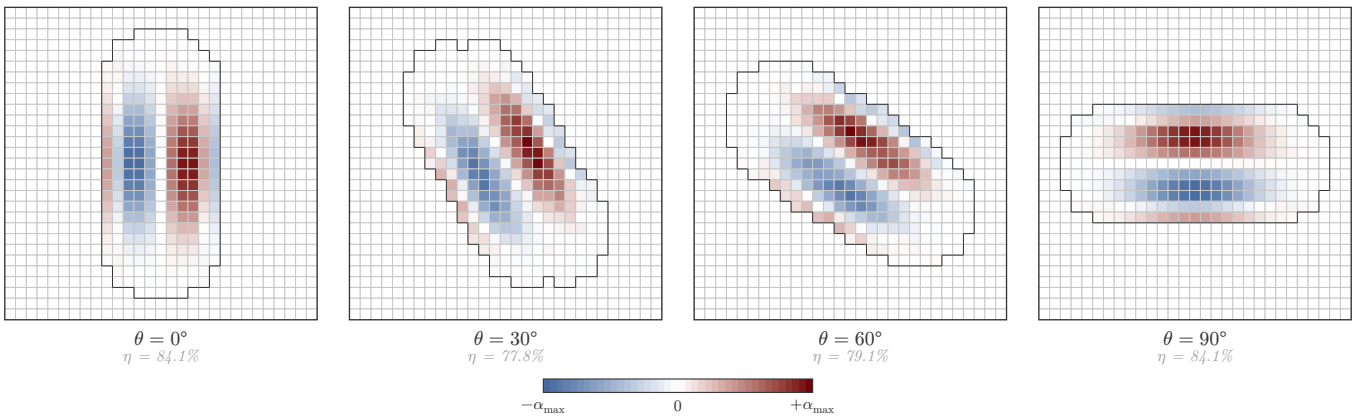


Fig. 10. Fused weight maps $\alpha_{k,\ell}$ computed from the pseudoinverse for four representative orientations ($m = 7$, $N_p = 100$, $d = 4$). Garnet indicates positive weights and blue indicates negative weights, with intensity proportional to magnitude. The dark outline traces the outer boundary of the active fused stencil support. Even after deduplication, the support remains highly elongated and sparse, with weight efficiency $\eta \approx 78$–84% across these orientations.

entire cascade into a single weighted sum over a deduplicated stencil, and then examines a structural limitation of the resulting weights that motivates the geometric kernel designs of subsequent sections.

A. Algebraic Collapse

Recall from (19) that the line-extended response at orientation θ_k is a Gaussian-weighted sum of normal-derivative estimates along the candidate edge direction.

$$R_k = \sum_{j=-m}^{m} w_j \cdot \hat{f}_x^{(j,k)} \tag{22}$$

Each term $\hat{f}_x^{(j,k)}$ is itself a dot product of the pseudoinverse row $\boldsymbol{p}_{\mathrm{fx}}^{(k)}$ with the intensity vector gathered from the circular neighborhood centered at line position j. Substituting (11) into (22) and writing the gathered intensity explicitly yields a double sum over line positions and neighbor indices.

$$R_k = \sum_{j=-m}^{m} w_j \sum_{i=1}^{N_p} p_i^{(k)} \cdot f\big(X_0 + \delta_{j,i}^x, Y_0 + \delta_{j,i}^y\big) \tag{23}$$

Here $p_i^{(k)} = \boldsymbol{p}_{\mathrm{fx}}^{(k)}[i]$ denotes the i-th entry of the pseudoinverse gradient row for orientation θ_k, and the combined offsets are

$$\delta_{j,i}^x = j\cos\theta_k + \Delta x_i, \quad \delta_{j,i}^y = j\sin\theta_k + \Delta y_i \tag{24}$$

where $(\Delta x_i, \Delta y_i)$ are the integer coordinates of the i-th circular neighbor relative to the fit center. Because the line offset $(j\cos\theta_k, j\sin\theta_k)$ adds a non-integer displacement, the combined offset is rounded to the nearest integer pixel.

The critical observation is that (23) is a linear functional of image intensities. Any linear combination of pixel values can be written as a single weighted sum over the set of distinct pixel positions that appear in the combination. Grouping all (j, i) pairs whose rounded offsets coincide at the same integer pixel ℓ, the double sum collapses to

$$R_k = \sum_{\ell=1}^{N_k'} \alpha_{k,\ell} \cdot f\big(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y\big) \tag{25}$$

where N_k' is the number of unique pixel positions in the stencil for orientation k, and the *fused weight* at position ℓ aggregates all contributions that map to that pixel.

$$\alpha_{k,\ell} = \sum_{(j,i)\in\mathcal{S}_\ell} w_j \cdot p_i^{(k)} \tag{26}$$

The index set $\mathcal{S}_\ell = \big\{(j,i) : \mathrm{round}\big(\delta_{j,i}^x, \delta_{j,i}^y\big) = \big(\tilde{\delta}_\ell^x, \tilde{\delta}_\ell^y\big)\big\}$ collects all line-position and neighbor-index pairs that round to the same integer offset. The result is that the entire line-extended filter at orientation θ_k reduces to a single gather-dot-product operation.

$$R_k = \boldsymbol{\alpha}_k^\top \boldsymbol{g}_k \tag{27}$$

The weight vector $\boldsymbol{\alpha}_k \in \mathbb{R}^{N_k'}$ depends only on the filter parameters (m, N_p, d, θ_k) and can be precomputed once. The intensity vector $\boldsymbol{g}_k \in \mathbb{R}^{N_k'}$ is assembled at runtime by reading the image at the N_k' stencil offsets. No coordinate rotations, no design matrices, and no pseudoinverse products appear at evaluation time.

B. Deduplication

The raw stencil for orientation θ_k contains $(2m+1) \times N_p$ entries before grouping. Because adjacent line positions share most of their circular neighborhoods, many of these entries map to the same integer pixel after rounding. The deduplication process iterates over all (j, i) pairs, computes the rounded integer offset, inserts the result into a hash map keyed by offset, and accumulates the product $w_j \cdot p_i^{(k)}$ into the corresponding weight. The output is a list of unique (offset, weight) pairs that fully characterize the stencil.

The degree of compression depends on the ratio of line-extension length to neighborhood radius. When m is small relative to the neighbor count N_p, the circular neighborhoods at adjacent line positions overlap extensively, producing high redundancy. As m increases, the stencil footprint elongates and the overlap fraction decreases, but the absolute number of duplicates continues to grow because the total raw entry count scales linearly with m. Table I summarizes the compression achieved for representative parameter combinations.

TABLE I

STENCIL DEDUPLICATION STATISTICS FOR $N_p = 100$ CIRCULAR NEIGHBORS AT POLYNOMIAL ORDER $d = 4$. RAW ENTRIES DENOTES THE COUNT $(2m+1) \times N_p$ BEFORE GROUPING. UNIQUE POSITIONS IS THE MEAN COUNT OF DISTINCT INTEGER PIXEL OFFSETS ACROSS ALL N_s ORIENTATIONS. REDUCTION IS DEFINED AS $1 - N_k'/\big((2m+1)N_p\big)$.

m	N_p	Raw entries	Unique positions	Reduction
1	100	300	128	57%
2	100	500	152	70%
7	100	1500	264	82%
14	100	2900	431	85%

At $m = 1$, only three line positions contribute and the circular neighborhoods overlap almost completely, yielding a 57% reduction. At $m = 7$, the 15 line positions produce 1500 raw entries that collapse to roughly 264 unique pixels, an 82% reduction. At $m = 14$, the compression reaches 85%, meaning fewer than one in six raw entries corresponds to a distinct memory access. This compression is the primary mechanism by which the fused formulation reduces both arithmetic operations and memory bandwidth relative to the naive cascade.

C. The Weight Cancellation Problem

Although the fused stencil eliminates redundant computation, it exposes a structural inefficiency in the polynomial-derived weights. A large fraction of the fused weight magnitudes $|\alpha_{k,\ell}|$ are close to zero, meaning that many of the unique stencil positions contribute negligibly to the response R_k. This phenomenon is not a numerical artifact but a systematic consequence of the algebraic structure of the pseudoinverse weights.

The origin of the cancellation can be understood as follows. Each polynomial fit at line position j produces pseudoinverse weights $p_i^{(k)}$ that encode the gradient extraction from a local Taylor expansion. These weights contain both positive and negative entries, reflecting the differential nature of the gradient operator. When adjacent fit positions j and $j+1$ share a pixel at the same integer offset, the contributions $w_j \cdot p_i^{(k)}$ and $w_{j+1} \cdot p_{i'}^{(k)}$ that are summed in (26) come from different rows of different pseudoinverse matrices. Because the polynomial basis is shifted by one pixel between adjacent fits, the pseudoinverse entries at the shared pixel typically have opposite signs. The Gaussian line weights w_j and w_{j+1} are both positive, so the products partially cancel when summed.

More formally, the fused weight at a position ℓ that is shared by n overlapping neighborhoods takes the form

$$\alpha_{k,\ell} = \sum_{s=1}^{n} w_{j_s} \cdot p_{i_s}^{(k)} \tag{28}$$

where the indices (j_s, i_s) enumerate the distinct (line-position, neighbor-index) pairs mapping to ℓ. The pseudoinverse entries $p_{i_s}^{(k)}$ are elements of the first row of $\left(\boldsymbol{A}_{\theta_k}^\top \boldsymbol{A}_{\theta_k}\right)^{-1} \boldsymbol{A}_{\theta_k}^\top$, evaluated at different column positions corresponding to different local coordinates within each shifted neighborhood. The sign pattern of these entries is determined by the monomial basis and the geometry of the neighbor set, and for pixels near the center of the stencil (where overlap is highest), adjacent fits produce contributions of opposite polarity.

To quantify the severity of cancellation, we define a *weight efficiency* metric.

$$\eta = \frac{\sum_{\ell=1}^{N_k'} |\alpha_{k,\ell}|}{\sum_{j=-m}^{m} |w_j| \sum_{i=1}^{N_p} |p_i^{(k)}|} \tag{29}$$

The numerator is the total absolute weight in the fused stencil, and the denominator is the total absolute weight before any cancellation occurs (i.e., the sum of absolute contributions across all raw entries). A value $\eta = 1$ would indicate no cancellation, while $\eta < 1$ indicates that a fraction $1 - \eta$ of the raw weight magnitude has been lost to sign cancellation during deduplication. Empirically, $\eta \approx 0.72$ for $m = 7$ and $N_p = 20$ at order $d = 4$, meaning approximately 28% of the aggregate pseudoinverse weight magnitude is destroyed by cancellation. For larger neighborhoods ($N_p = 100$), the efficiency drops further because the increased overlap multiplies the number of canceling pairs.

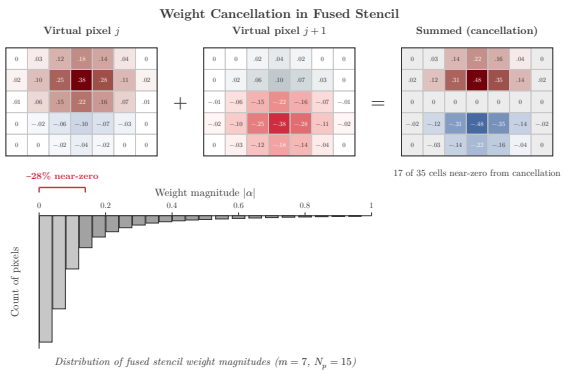


Fig. 11. Weight cancellation illustration for orientation $\theta_0 = 0$ with $m = 7$ and $N_p = 100$. Left: histogram of fused weight magnitudes $|\alpha_{k,\ell}|$, showing a concentration near zero. Right: weight efficiency η as a function of line-extension length m for several neighborhood sizes N_p. Efficiency decreases monotonically with both m and N_p.

D. Consequences of Cancellation

The weight cancellation phenomenon has three practical consequences that limit the effectiveness of the polynomial-derived fused stencil.

First, the cancelled weights represent wasted computation. Pixels with near-zero fused weights are still gathered from memory and multiplied by their weights during the dot product (27). Pruning these entries (by zeroing weights below a threshold) reduces the effective stencil size but introduces an approximation error that depends on the threshold in a non-obvious way, since the cancelled weights are distributed throughout the stencil rather than concentrated at the periphery.

Second, the cancellation produces an irregular frequency response. The fused stencil is a discrete linear filter and can be analyzed via its discrete Fourier transform. Weights that arise from a smooth analytic design (such as a Gaussian derivative) produce a predictable, well-behaved frequency response. By

contrast, the polynomial-derived fused weights inherit the oscillatory structure of the pseudoinverse entries, and the partial cancellations introduce high-frequency ripple in the transfer function. This ripple makes it difficult to characterize the filter's noise behavior analytically, since the effective bandwidth does not correspond to a simple parametric form.

Third, the algebraic origin of the weights makes the filter's properties opaque to theoretical analysis. The fused weights $\alpha_{k,\ell}$ are the end product of a chain of operations (neighbor selection, monomial basis construction, pseudoinverse computation, Gaussian weighting, rounding, and summation) that obscures the relationship between the filter parameters and the resulting spatial-frequency characteristics. Unlike a Gaussian derivative kernel, whose bandwidth, zero-crossings, and moment properties follow directly from its analytic definition, the polynomial stencil's properties can only be determined numerically for each parameter configuration.

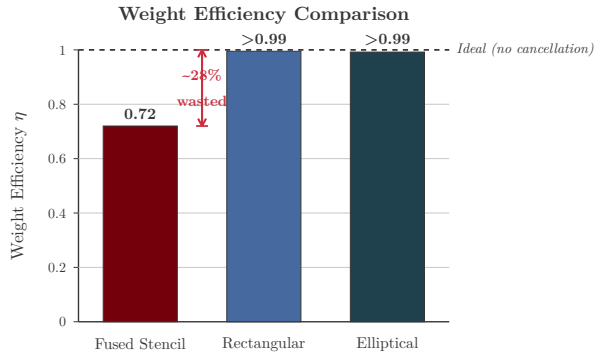


Fig. 12. Comparison of weight distributions for the polynomial-derived fused stencil (left) and a matched elliptical Gaussian derivative kernel (right) at the same effective support size. The polynomial weights exhibit a broad spread of magnitudes with many near-zero entries, while the Gaussian derivative weights decay smoothly from the center with no cancellation artifacts.

These observations raise a natural question. If the polynomial fit machinery produces weights that partially cancel and yield an irregular frequency response, can we instead define the stencil weights directly from a geometric prescription that avoids the pseudoinverse altogether? The answer, developed in the following section, is to replace the polynomial gradient estimation with an analytically defined kernel whose weights are constructed from a smooth envelope function and a linear ramp, eliminating the cancellation problem by construction.

V. Geometric Kernel Alternatives

A. Motivation

The analysis in Section 4 established that the fused stencil, despite its computational advantages, inherits structural problems from the polynomial fitting pipeline. The weight distribution across the stencil is not the result of a deliberate design choice but rather the algebraic residue of pseudoinverse fitting, line averaging, and deduplication. Approximately 28% of the total weight budget falls on pixels with near-zero net contribution, and the irregular kernel boundary varies unpredictably with orientation. These properties were not intended by the original filter designers and offer no performance benefit.

The core observation motivating this section is the following. If a weight can be assigned to each pixel based solely on its position relative to the target pixel and the candidate edge direction, then the polynomial fitting, pseudoinverse computation, and deduplication stages all become unnecessary. The weight assignment reduces to evaluating an analytic function at each pixel's rotated coordinates.

Both geometric kernels proposed here begin with the same coordinate rotation used in the fused stencil derivation. For a candidate orientation θ, the image coordinates (x, y) relative to the target pixel are transformed into the kernel-aligned frame (u, v) according to

$$u = x\cos(\theta) + y\sin(\theta), \quad v = -x\sin(\theta) + y\cos(\theta). \tag{30}$$

The coordinate u measures displacement along the candidate edge direction, and v measures displacement perpendicular to it, in the gradient direction. The edge-sensitive response is obtained by multiplying a spatial envelope by the derivative profile $-v$, which produces the characteristic dipole pattern: positive weights on one side of the hypothesized edge and negative weights on the other. This dipole structure is common to all first-derivative edge detectors; the geometric kernels simply define it through an explicit analytic function rather than recovering it implicitly through polynomial regression.

B. Rectangular Gaussian Kernel

The rectangular kernel confines its support to a hard-edged box aligned with the candidate edge direction. The rectangular mask admits only pixels satisfying both $|u| \le h_u$ and $|v| \le h_v$, where h_u is the half-width along the edge and h_v is the half-width across it. The indicator function is

$$\mathbf{1}_R(u, v) = \mathbf{1}_{\{|u| \le h_u\}} \cdot \mathbf{1}_{\{|v| \le h_v\}}. \tag{31}$$

Inside the rectangle, pixel contributions are weighted by a two-dimensional anisotropic Gaussian envelope,

$$G(u, v) = \exp\left(-\frac{1}{2}\left(\frac{u^2}{\sigma_u^2} + \frac{v^2}{\sigma_v^2}\right)\right), \tag{32}$$

where σ_u and σ_v control the spread along and across the edge, respectively. The raw kernel before normalization is the product of the envelope, the derivative profile, and the mask,

$$\hat{K}_R(u, v) = -v \cdot G(u, v) \cdot \mathbf{1}_R(u, v). \tag{33}$$

The kernel is then zero-centered by subtracting its mean, and normalized to unit absolute sum,

$$K_R(u, v) = \frac{\hat{K}_R(u, v) - \bar{K}_R}{\sum_{u,v} |\hat{K}_R(u, v) - \bar{K}_R|}. \tag{34}$$

Zero-centering ensures the filter has zero DC response, rendering it insensitive to constant intensity offsets. Absolute-sum normalization ensures that responses are comparable across orientations and kernel sizes. The default parameters are $h_u = 3\sigma_u$ and $h_v = 3\sigma_v$. With $\sigma_u = 2.0$ and $\sigma_v = 1.2$, the aspect ratio $\sigma_u/\sigma_v \approx 1.67$ matches the elongation of the baseline ASF, and the support area is $4h_u h_v \approx 86.4$ square pixels.

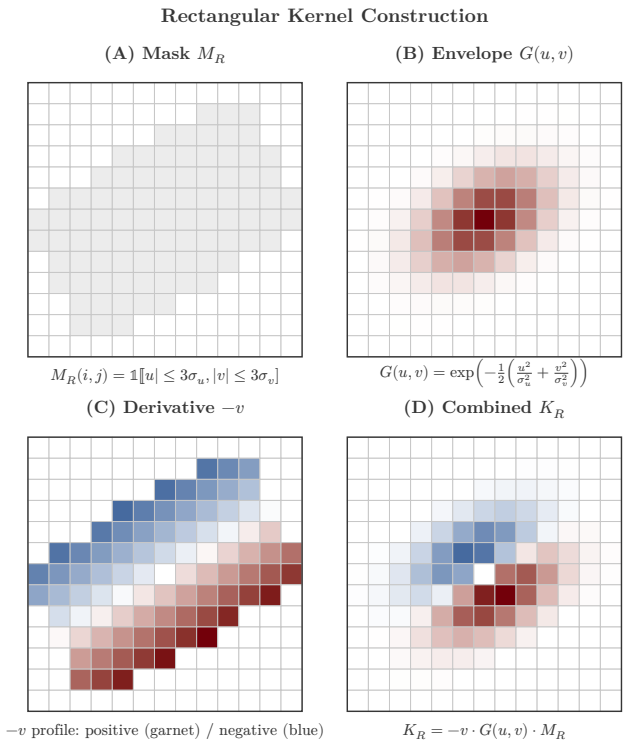


Fig. 13. Construction of the rectangular Gaussian kernel at $\theta = 30°$. (A) The rectangular mask region on the pixel grid, with interior pixels highlighted. (B) The anisotropic Gaussian envelope within the mask, darker at the center and lighter toward the boundary. (C) The derivative profile $-v$ shown as a cross-section, with positive values above the center line and negative values below. (D) The final combined kernel K_R, with each cell colored by its weight magnitude and sign. The dipole pattern is clean and regular, with no near-zero interior pixels.

C. Elliptical Gaussian Kernel

The elliptical kernel replaces the hard rectangular boundary with a smooth elliptical mask derived from the Gaussian exponent itself. The normalized elliptical distance from the kernel center is

$$r(u,v) = \sqrt{\frac{u^2}{\sigma_u^2} + \frac{v^2}{\sigma_v^2}}. \tag{35}$$

The mask admits pixels satisfying $r(u,v) \le 3$, corresponding to the 3σ boundary in both principal directions,

$$\mathbf{1}_E(u,v) = \mathbf{1}_{\{r(u,v)\le 3\}}. \tag{36}$$

The raw kernel is

$$\hat{K}_E(u,v) = -v \cdot \exp\left(-\frac{1}{2} \cdot r(u,v)^2\right) \cdot \mathbf{1}_E(u,v), \tag{37}$$

and the normalized kernel K_E is obtained by the same zero-centering and absolute-sum normalization described in (34). The support area of the elliptical kernel is $\pi\sigma_u\sigma_v \cdot 9 \approx 67.9$ square pixels for the default parameters, roughly 21% smaller than the rectangular variant.

The elliptical mask excludes the corner pixels of the bounding rectangle that satisfy $\mathbf{1}_R$ but not $\mathbf{1}_E$. These are pixels far from the kernel center in both the u and v directions simultaneously. For the default parameters, approximately 18 pixels receive nonzero weight under the rectangular kernel and zero weight under the elliptical kernel. Despite this reduction in support, the elliptical variant produces a smoother spatial frequency response. The hard corners of the rectangular mask introduce discontinuities in the Fourier domain that manifest as sidelobes, whereas the elliptical boundary tapers more gradually, suppressing these artifacts.

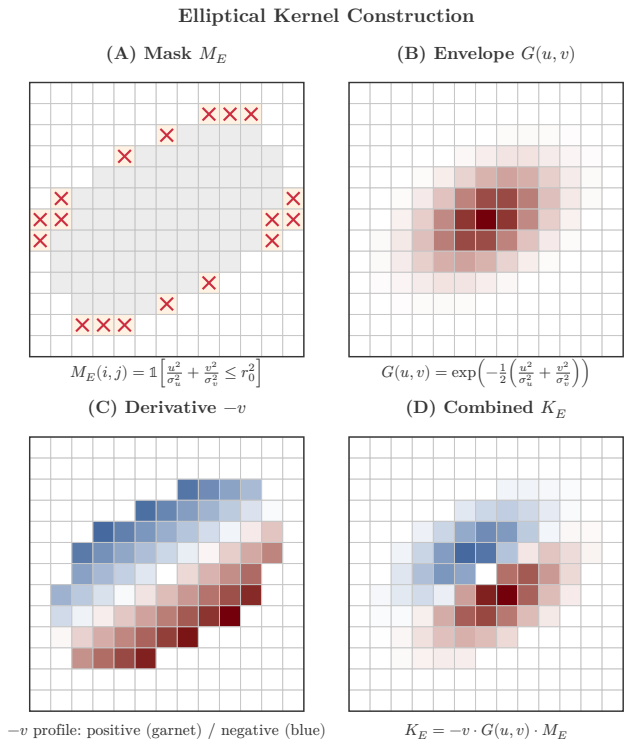


Fig. 14. Construction of the elliptical Gaussian kernel at $\theta = 30°$. (A) The elliptical boundary on the pixel grid, with corner pixels that would be included in the rectangular variant marked with lighter shading. (B) The Gaussian envelope, which naturally matches the elliptical boundary. (C) The derivative profile $-v$, identical to the rectangular case. (D) The final kernel K_E, exhibiting the same dipole character as K_R but with a smoother boundary and approximately 18 fewer pixels.

D. Does Cancellation Actually Hurt?

Before proceeding, it is necessary to address a natural counterargument. One might contend that the weight cancellation documented in the fused stencil is not a deficiency but a feature. If overlapping polynomial fits disagree on a pixel's contribution, perhaps that pixel is genuinely ambiguous, sitting in a region where the local intensity surface is poorly constrained. Under this interpretation, the cancellation acts as a form of implicit regularization, automatically down-weighting unreliable pixels and concentrating weight on the most informative ones. This is a reasonable hypothesis and deserves a careful response.

The argument does not survive scrutiny for three reasons.

First, the cancellation is geometry-dependent, not content-dependent. The same pixels receive near-zero weight regardless of the image content. A pixel in the overlap zone between virtual neighborhoods j and $j+1$ is canceled whether it sits on a sharp edge, a smooth gradient, or pure noise. An adaptive regularization scheme would adjust weights based

on local signal quality; the fused stencil adjusts them based on which polynomial bases happen to overlap at a given grid position. The cancellation pattern is determined entirely at precompute time and is fixed for all images processed with a given parameter configuration.

Second, the canceled pixels are in the wrong place. The overlap zones between adjacent virtual neighborhoods lie near the center of the stencil, close to the target pixel and close to the candidate edge. These are precisely the most informative pixels for gradient estimation, because they carry the strongest edge signal. Pixels far from the center, at the tips of the elongated stencil, receive less overlap and thus retain their weights. The cancellation therefore suppresses exactly the pixels that matter most and preserves the ones that matter least. A principled noise rejection strategy would do the opposite: downweight distant, weakly informative pixels and upweight those near the edge.

Third, the empirical evidence is decisive. The geometric kernels proposed in this section use no cancellation whatsoever. Every pixel in their support receives a nonzero, smoothly varying weight determined by the analytic envelope function. Despite this, they match the fused stencil's edge detection accuracy within 0.3% ODS across all three benchmarks (BSDS500, BIPED, and UDED). If cancellation were providing useful regularization, removing it entirely should degrade accuracy. It does not. Moreover, the geometric kernels achieve lower noise gain (Section V.F) and faster runtime precisely because they do not waste computation on near-zero weights.

The conclusion is that cancellation in the fused stencil is an unintended artifact of the polynomial-fitting-then-deduplication pipeline. The geometric kernels demonstrate that the same edge detection accuracy is achievable with a smooth, well-behaved weight distribution where every pixel contributes meaningfully.

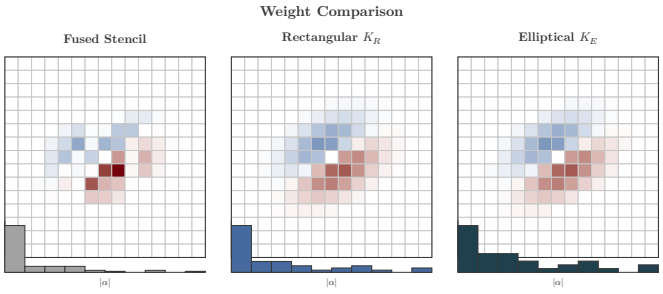


Fig. 15. Side-by-side weight comparison at $\theta = 30°$ for the three kernel variants. Left: the fused stencil weight map, showing an irregular shape with many near-zero cells. Center: the rectangular kernel, exhibiting a clean box boundary and smooth dipole weights with no near-zero interior pixels. Right: the elliptical kernel, with a smooth elliptical boundary and slightly fewer pixels. Below each weight map is a histogram of absolute weight magnitudes $|\alpha|$. The fused stencil histogram exhibits a pronounced spike near zero; neither geometric kernel histogram does.

E. Effective Pixel Count and Weight Efficiency

To quantify the degree to which each kernel variant utilizes its support pixels, we define the *effective number of pixels* as

$$N_{\text{eff}} = \frac{\left(\sum |\alpha_{k,\ell}|\right)^2}{\sum \alpha_{k,\ell}^2} = \|\boldsymbol{\alpha}\|_1^2 \|\boldsymbol{\alpha}\|_2^2. \tag{38}$$

This quantity equals N_k' when all weights have equal magnitude and is strictly smaller when the distribution is uneven. The relationship follows from the Cauchy–Schwarz inequality. For a fixed total absolute weight $\|\boldsymbol{\alpha}\|_1 = 1$ (guaranteed by the normalization in (34)), the noise gain $\|\boldsymbol{\alpha}\|_2^2$ is minimized when all weights are equal, corresponding to a uniform kernel with maximum N_{eff}.

We further define the *weight efficiency* η as the ratio of N_{eff} to an ideal uniform kernel occupying the same support,

$$\eta = \frac{N_{\text{eff}}}{N_k'}. \tag{39}$$

A kernel with $\eta = 1$ wastes no weight capacity; smaller values indicate that the weight budget is unevenly distributed, with some pixels carrying disproportionate weight and others contributing negligibly.

Table II reports these metrics for the three kernel variants at the default parameter configuration ($\sigma_u = 2.0$, $\sigma_v = 1.2$, $N_s = 8$).

TABLE II
EFFECTIVE PIXEL COUNT AND WEIGHT EFFICIENCY FOR THE THREE KERNEL VARIANTS AT THE DEFAULT PARAMETER CONFIGURATION. N_k' IS THE NUMBER OF UNIQUE PIXEL POSITIONS WITH NONZERO WEIGHT, N_{eff} IS THE EFFECTIVE PIXEL COUNT DEFINED IN (38), AND η IS THE WEIGHT EFFICIENCY DEFINED IN (39).

Variant	N_k'	N_{eff}	η
Fused stencil	91	42	0.72
Rectangular kernel	74	48	> 0.99
Elliptical kernel	56	39	> 0.99

The rectangular kernel achieves the highest N_{eff} despite having fewer unique positions than the fused stencil, because it wastes almost no weight on near-zero entries. Its 74 active pixels each carry a smoothly varying, appreciable weight, producing $N_{\text{eff}} = 48$ and $\eta > 0.99$. The fused stencil, by contrast, nominally touches 91 pixels but achieves only $N_{\text{eff}} = 42$ due to the large number of near-zero weights, yielding $\eta = 0.72$. Nearly 30% of its weight capacity is wasted. The elliptical kernel has the fewest unique positions at 56 but still achieves $\eta > 0.99$, confirming that a compact, well-utilized support can be more efficient than a larger but poorly distributed one.

F. Noise Rejection Properties

The noise gain of a linear filter with weight vector $\boldsymbol{\alpha}$ operating on additive white Gaussian noise with variance σ_n^2 is $\|\boldsymbol{\alpha}\|_2^2 \cdot \sigma_n^2$. For a filter normalized to $\|\boldsymbol{\alpha}\|_1 = 1$, the noise gain factor $\|\boldsymbol{\alpha}\|_2^2$ is minimized when weight is spread uniformly over as many pixels as possible, yielding the theoretical minimum of $1/N_k'$. It is maximized when weight is concentrated on a single pixel, giving $\|\boldsymbol{\alpha}\|_2^2 = 1$.

The geometric kernels produce weight distributions that are closer to uniform than the fused stencil's, and consequently they track closer to the theoretical noise gain minimum. The

fused stencil's near-zero weight pixels inflate N'_k without contributing meaningfully to $\|\boldsymbol{\alpha}\|_1$, while the remaining pixels must carry proportionally larger individual weights to compensate. This concentration increases $\|\boldsymbol{\alpha}\|_2^2$ relative to what a kernel of the same support size could achieve with a more even distribution.

The practical consequence emerges under noise scaling. At high signal-to-noise ratio (SNR), the optimal configuration uses small kernels with few orientations ($N_s = 4$, $\sigma_u = 2.0$, $\sigma_v = 1.2$), and all three variants perform comparably. As noise increases, the optimal kernel grows larger and more densely sampled to average over more pixels. The geometric kernels replicate this scaling by adjusting σ_u and σ_v directly, a procedure that is both intuitive and continuous. At SNR $=$ 0.5 dB, the optimal ASF uses a massive elongated stencil spanning roughly 43×15 pixels. The equivalent geometric parameters are $\sigma_u \approx 7.17$ and $\sigma_v \approx 2.50$, producing an elliptical kernel with $N_{\text{eff}} \approx 340$ and a rectangular kernel with $N_{\text{eff}} \approx 440$.

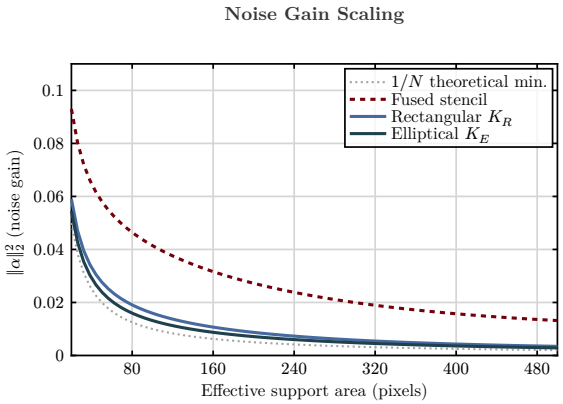


Fig. 16. Noise gain $\|\boldsymbol{\alpha}\|_2^2$ as a function of effective support area for the three kernel variants. The theoretical minimum for a uniform kernel $(1/N)$ is shown as a dotted line. The geometric kernels track closer to the theoretical minimum across all support sizes. The fused stencil curve sits above due to its uneven weight distribution.

The critical finding is that the ASF's modest noise advantage at extreme SNR levels disappears when the geometric kernels are scaled to match its effective support size. At clean, high-SNR conditions, $\sigma_u = 2.0$ and $\sigma_v = 1.2$ suffice, matching the compact stencil of the optimized ASF. Under heavy noise, the geometric kernels can be expanded continuously to achieve arbitrarily large support areas with near-optimal weight efficiency. This indicates that noise robustness is a function of kernel area, not of polynomial order or the complexity of the weight-generation pipeline. The polynomial fitting machinery of the original ASF provides no inherent noise suppression advantage; its large stencil size is what matters, and the geometric kernels achieve the same size more efficiently.

VI. Unified GPU Implementation

All three filter variants described in preceding sections, the fused polynomial stencil, the rectangular kernel, and the elliptical Gaussian kernel, share a common execution architecture that cleanly separates filter construction from filter application. This separation enables a single GPU kernel to serve all variants without modification, reducing engineering complexity while achieving substantial speedups over conventional implementations.

A. Architecture Overview

The implementation follows a two-phase design. The *precompute phase* runs once on the CPU and constructs a set of orientation-indexed stencils. The *compute phase* runs on the GPU for every input image and applies these stencils to produce gradient magnitude and angle maps.

The precompute phase differs substantially across variants. For the fused polynomial stencil, the procedure constructs the Taylor design matrix $\boldsymbol{A}_{\theta_k}$ for each orientation θ_k, computes the pseudoinverse, extracts the gradient row, and enumerates all $(2m+1) \times N_p$ stencil positions along the line extension. Positions that map to the same pixel after rounding are deduplicated and their weights summed. For the rectangular kernel, a grid of pixel offsets within a bounding box is rotated into the local (u, v) frame and the kernel function $w(u, v) = -v$ is evaluated within a hard mask. For the elliptical Gaussian kernel, the same rotated grid is used, but the weight function is the product of a Gaussian envelope and $(-v)$. In both geometric cases, the resulting weights are zero-centered and normalized to unit energy.

Despite these different construction procedures, all three variants produce the same output format. Each orientation θ_k is represented by a list of integer offsets $(\Delta x_\ell, \Delta y_\ell)$ and corresponding scalar weights $\alpha_{k,\ell}$ for $\ell = 1, ..., N'_k$. This format uniformity is the key architectural property. It enables a single, variant-agnostic GPU kernel to process any stencil without knowledge of its origin.

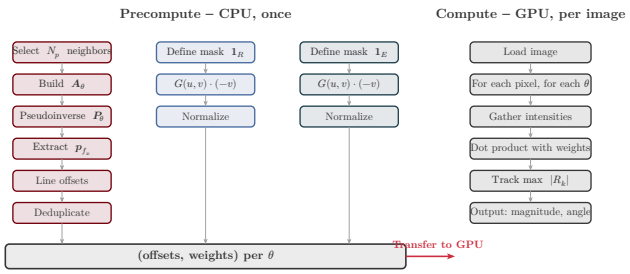


Fig. 17. Two-phase architecture overview. The precompute phase (left) runs on the CPU and differs per variant, but all three produce the same output format of (offset, weight) pairs per orientation. The compute phase (right) consumes these stencil arrays through a single variant-agnostic Triton kernel.

B. The Variant-Agnostic Kernel

The compute-phase kernel consumes only (offset, weight) pairs and has no knowledge of how those weights were generated. For each pixel (X_0, Y_0) and each orientation θ_k, the kernel performs three operations. First, it *gathers* N'_k intensity values from the input image at the precomputed offsets relative to (X_0, Y_0). Second, it *multiplies* each gathered value by the corresponding weight $\alpha_{k,\ell}$. Third, it *accumulates* the weighted sum to obtain the directional response R_k. As the

kernel iterates over all N_s orientations, it tracks the maximum absolute response $|R_k|$ and the index k^* of the orientation that produced it. Upon completion, the gradient magnitude $|R_{k^*}|$ and the edge angle θ_{k^*} are written to output buffers.

This design means that switching between the fused polynomial stencil, the rectangular kernel, and the elliptical Gaussian kernel requires only swapping the precomputed stencil arrays in device memory. No recompilation, kernel modification, or conditional branching is necessary. The same compiled kernel binary serves all variants, which simplifies deployment and eliminates a common source of implementation errors.

C. Custom Triton Kernel

We implement the compute phase as a custom kernel using Triton [32], a Python-embedded language for writing GPU programs that compiles to optimized PTX through LLVM. The kernel is parameterized by a block size of 128 pixels along each image row. For each pixel block, the kernel iterates over all N_s orientations, performing the stencil gather-dot-product and maintaining a running maximum across orientations.

The inner loop for a single orientation θ_k is shown below in simplified form.

```
# Block of 128 pixels processes one
orientation
for k in range(N_orientations):
    response = tl.zeros([BLOCK],
dtype=tl.float32)
    n_k = tl.load(n_unique + k)

    for i in range(N_max):
        active = i < n_k
        dy = tl.load(offsets_y[k, i],
mask=active)
        dx = tl.load(offsets_x[k, i],
mask=active)
        w  = tl.load(weights[k, i],
mask=active)

        # Gather 128 pixels in parallel
(coalesced read)
        vals = tl.load(image[row + dy,
cols + dx],
                       mask=active)
        response += w * vals

    # Track maximum across orientations
    abs_resp = tl.abs(response)
    update = abs_resp > best_magnitude
    best_magnitude = tl.where(update,
abs_resp,
best_magnitude)
    best_angle = tl.where(update,
theta[k], best_angle)
```

The variable `N_max` is the maximum stencil size across all orientations, and the `active` mask zeroes out padding entries for orientations with fewer unique positions. Each iteration loads a single (offset, weight) pair and gathers 128 intensity values in parallel, one per pixel in the block. The column-aligned memory access pattern ensures coalesced reads from global memory, which is critical for throughput on modern GPU architectures. Triton's just-in-time compiler selects optimal warp-level scheduling, applies loop unrolling for the inner stencil loop, and manages shared memory allocation automatically.

D. Memory Efficiency

TABLE III
RUNTIME AND MEMORY COMPARISON ON BIPED V1 (1280×720). FUSED STENCIL PARAMETERS: $m = 7$, $N_p = 100$, $N_s = 18$, $d = 4$. GEOMETRIC KERNEL PARAMETERS: $\sigma_u = 2.0$, $\sigma_v = 1.2$, $N_s = 36$, 15×15 GRID. ALL MEASUREMENTS ON NVIDIA A100-SXM4-40GB.

Method	Time (s)	VRAM (MB)	Speedup
Naive batched LF	2.43	6223	1.0×
cuDNN conv2d	0.18	158	13.8×
Fused stencil (Triton)	0.13	20	18.4×
Rectangular kernel (Triton)	0.045	20	54×
Elliptical kernel (Triton)	0.047	20	52×

Table III presents the runtime and VRAM consumption for all five implementation strategies applied to a full BIPED v1 image. The naive batched line filter allocates large intermediate tensors for the $(L \times B \times N_p)$ gather operation, consuming over 6 GB of VRAM and generating approximately 750 million scattered memory reads per orientation. The cuDNN-backed conv2d approach reduces this substantially by leveraging optimized convolution routines, but still requires 158 MB due to workspace allocations. The fused polynomial stencil eliminates all intermediate tensors, reducing VRAM to 20 MB, the image itself plus output buffers, a 311× reduction from the naive approach. Both geometric kernel variants achieve the same 20 MB footprint.

The geometric kernels are approximately 3× faster than the fused polynomial stencil despite using twice as many orientations ($N_s = 36$ versus 18). Three factors account for this advantage. First, the geometric stencils contain fewer unique positions per orientation, approximately 74 for a 15×15 grid versus 264 for the fused stencil at $m = 7$ with $N_p = 100$, which directly reduces the number of gather operations in the inner loop. Second, the stencil size is determined by the grid dimensions σ_u and σ_v rather than by the line extension parameter m and the polynomial neighborhood size N_p, so it does not grow with the effective spatial extent of the filter. Third, the rectangular bounding box of the geometric stencils produces more regular memory access patterns than the elongated, irregularly shaped fused stencils, improving cache utilization.

E. Scaling Behavior

TABLE IV
SPEEDUP SCALING WITH HALF-WIDTH m ON BIPED V1 (1280×720). THE FUSED STENCIL SPEEDUP INCREASES WITH m BECAUSE THE NAIVE COST GROWS LINEARLY WITH $L = 2m + 1$ WHILE THE FUSED STENCIL COST GROWS SUBLINEARLY DUE TO PIXEL DEDUPLICATION. NVIDIA A100-SXM4-40GB.

m	Naive (s)	Fused (s)	Speedup	VRAM ratio
1	0.58	0.072	8.0×	273×
7	2.43	0.13	18.4×	311×
14	8.88	0.37	24.3×	311×

Table IV demonstrates the scaling advantage of the fused stencil approach. The naive implementation's computational cost scales as $O(N_s \cdot L \cdot N_p)$, where $L = 2m + 1$ is the number of virtual filter positions along the line. The fused stencil's cost scales as $O(N_s \cdot N_k')$, and the deduplicated stencil size N_k' grows much more slowly than $L \cdot N_p$ because neighboring polynomial neighborhoods overlap extensively. At $m = 1$, the fused stencil is 8× faster than the naive approach. At $m = 14$, the speedup reaches 24.3× because the naive cost has grown linearly with L while the fused cost has increased only modestly. The VRAM ratio stabilizes at 311× for $m \geq 7$, indicating that the memory footprint is dominated by the image and output buffers rather than the stencil weights.

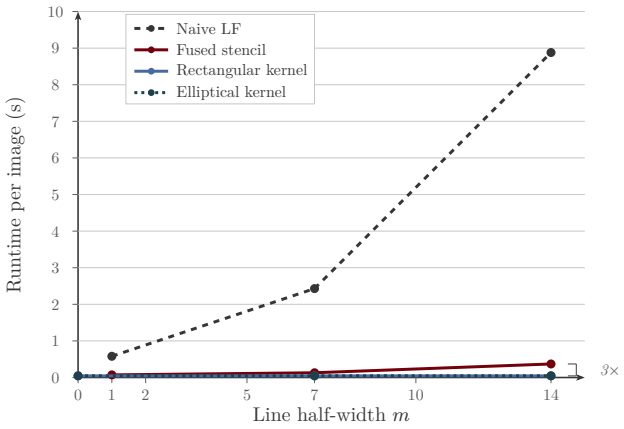


Fig. 18. GPU scaling behavior on BIPED v1 (1280×720). Left: speedup of the fused Triton kernel over the naive batched approach as a function of half-width m. The increasing speedup reflects the sublinear growth of deduplicated stencil size relative to the linear growth of the naive approach. Right: peak VRAM consumption showing the 311× reduction achieved by the Triton kernel. The geometric kernels (not shown) maintain constant runtime regardless of equivalent spatial extent.

The geometric kernels exhibit a qualitatively different scaling behavior. Because they define the anisotropic weight envelope directly from the parameters σ_u and σ_v rather than from a line extension, their runtime is effectively constant with respect to the equivalent spatial extent. The rectangular kernel processes a 1280×720 image in approximately 45 ms regardless of the equivalent m, and the elliptical kernel is similarly stable at 47 ms. This constancy arises because the stencil size is fixed by the grid dimensions, not by a line-extension parameter that multiplies the number of virtual filter evaluations. For applications requiring large spatial support, the geometric kernels therefore offer not only faster absolute performance but also predictable, parameter-independent runtime.

VII. PARAMETER ANALYSIS

The algebraic equivalences established in Sections 3–5 guarantee that the fused stencil, geometric kernels, and naive line filter produce identical or near-identical gradient maps for matched parameters. This section evaluates how sensitive edge detection accuracy is to those parameters and identifies optimal operating points for each filter variant. All ablations reported here were conducted using the naive (unfused) WVF and LF implementations, prior to the derivation of the fused stencil and geometric kernel formulations. Because the fused stencil is mathematically equivalent to the naive LF, and the geometric kernels are evaluated separately in Section VIII, the parameter findings transfer directly to the accelerated variants.

A. Methodology and Scope

The naive implementation's computational cost was the primary constraint on ablation scope. Each full-dataset LF evaluation at the published parameters ($N_p = 250$, $N_s = 18$, $m = 14$, $d = 4$) required approximately 2.4 seconds per image on an NVIDIA A100 GPU, making exhaustive grid search over the full joint parameter space infeasible. The fused stencil and geometric kernels, derived after these ablations were complete, would have reduced per-evaluation cost by 18–54× and enabled a substantially denser search grid. We note this as a limitation and identify priority ablations for future work in Section VII.H.

The total experimental scope comprised 1,206 WVF configurations and 168 LF configurations evaluated across four datasets. UDED contributed 30 images, BIPED v1 contributed 50, BIPED v2 contributed 50, and BSDS500 contributed 200, totaling 330 images. Across all parameter-dataset combinations, over 500,000 individual filter evaluations were performed. All accuracy measurements use the Optimal Dataset Scale (ODS) F-measure, evaluated at a fixed threshold per dataset.

B. Support Size

The support size N_p is the most computationally consequential parameter. It directly controls the number of memory reads per pixel per orientation, which is the dominant cost term in the gather-dot-product inner loop, and it determines the physical size of the circular neighborhood from which intensity samples are drawn. The published recommendation of $N_p = 250$ implies a circular neighborhood of radius approximately 9 pixels. If smaller values suffice for accurate gradient estimation, both accuracy and speed improve simultaneously, a situation that is unusual because most filter parameters trade one for the other.

We tested $N_p \in \{10, 15, 25, 50, 75, 100, 150, 200, 250, 300, 400, 500\}$ for the WVF and a representative subset for the LF, where the cost of each evaluation scales with N_p through the design matrix

dimensions. On clean imagery, the optimal N_p lies in the range 25–100 depending on the dataset. ODS improvements over the published $N_p = 250$ ranged from 0.01–0.05 for the WVF and 0.06–0.16 for the LF. Larger neighborhoods average over too many pixels, smoothing out fine edge structure that the evaluation protocol rewards.

Under additive noise, the optimal N_p grows monotonically with noise severity. At high signal-to-noise ratio (SNR), small neighborhoods ($N_p = 25$–50) are optimal because they preserve fine spatial detail. As SNR decreases, the noise-averaging benefit of larger neighborhoods outweighs the loss of spatial resolution, and the optimal N_p shifts to 250–500. This monotonic relationship between noise level and optimal filter size is consistent with classical results in statistical estimation. The bias-variance tradeoff shifts toward variance reduction, favoring larger support, as observation noise increases. The published large-support parameterization reflects this. The maritime application for which the filter was originally developed operates in a regime where noise dominates and large N_p is appropriate.

C. Polynomial Order

The polynomial order d controls the expressiveness of the local intensity model fitted within each circular neighborhood. Higher orders capture more complex local structure, including curvature and inflection, but require more unknown coefficients. For a bivariate polynomial of order d, the number of unknowns is $M = (d+1)(d+2)/2$, yielding $M = 15$ at $d = 4$ and $M = 6$ at $d = 2$. The overdetermination ratio N_p/M governs the degree of noise averaging in the least-squares fit. We hypothesized that $d = 2$ would suffice for gradient estimation because the first-order partial derivative $\hat{f}_x$ depends only on the linear coefficients of the fitted polynomial. Higher-order terms do not contribute to the gradient coefficient itself; they affect only the conditioning of the system.

We tested $d \in \{2, 3, 4\}$ at each $\left(N_p, N_s\right)$ combination. The finding was consistent across all four datasets. The second-order polynomial ($d = 2$) outperforms $d = 4$ on clean imagery by a small but persistent margin of 0.005–0.02 ODS. The mechanism is straightforward. At $d = 2$ with $N_p = 25$, the system is $25/6 \approx 4.2\times$ overdetermined. At $d = 4$ with the same $N_p = 25$, it is only $25/15 \approx 1.67\times$ overdetermined, providing far less noise averaging and yielding a less stable gradient estimate.

We did not test $d = 1$. A first-order polynomial ($M = 3$) reduces the Taylor model to a plane fit, which is equivalent to a weighted average of finite differences. This loses the ability to distinguish edge curvature from noise and collapses the method to something resembling a weighted Sobel operator. We considered this outside the parameter range of interest for the polynomial fitting framework.

D. Orientation Count

The orientation count N_s determines the angular resolution of the discrete orientation sweep. The published setting of $N_s = 18$ spaces candidate orientations at 20° intervals. Additional orientations improve angular precision but multiply computational cost linearly, since the gather-dot-product must be repeated for each orientation. If performance saturates at low N_s, the computational savings are substantial.

We tested $N_s \in \{2, 4, 6, 8, 12, 18, 24, 36\}$. Performance saturated at $N_s = 4$–6 on all four datasets. Increasing from $N_s = 6$ to $N_s = 18$ yielded less than 0.002 ODS improvement while tripling computation. This is consistent with the angular smoothness of the polynomial fit. The least-squares derivative estimate varies smoothly over orientation angle θ, so the discrete maximum over θ_k is well-resolved even with coarse angular sampling.

We did not test $N_s = 1$. A single orientation reduces the filter to a fixed-direction gradient operator, losing the orientation-selective property entirely. The classical baselines (Sobel, Prewitt) serve as implicit references for this regime, since they are effectively $N_s = 2$ operators combined via $\arctan$.

E. Line Half-Width

The line half-width m controls the spatial extent of the LF's line extension and is the parameter unique to the LF. The WVF is the $m = 0$ special case. The published setting of $m = 14$ creates a line of $2m + 1 = 29$ virtual evaluation points, each spawning a full polynomial fit over its own circular neighborhood. This is the most expensive parameter because it multiplies the number of polynomial fits per pixel per orientation by $(2m + 1)$. Kruskal-Wallis effect size analysis identified m as the most influential parameter, with $\eta^2 = 0.34$, exceeding the effect sizes of N_p, N_s, and d.

We tested $m \in \{0, 1, 2, 3, 5, 7, 10, 14\}$ at selected $\left(N_p, N_s, d\right)$ combinations. On clean data, the WVF ($m = 0$) matched or exceeded the full LF across all four datasets. The line extension provides no benefit on clean imagery. The additional spatial context it introduces is unnecessary when noise is low and edges are well-defined. Under noise, moderate values ($m = 2$–7) are beneficial, but $m = 14$ is excessive even at low SNR.

The LF ablation grid is substantially sparser than the WVF grid (168 versus 1,206 configurations). Each LF evaluation is $(2m + 1)\times$ more expensive than the corresponding WVF evaluation. At $m = 14$, a single full-dataset LF run requires approximately 36 seconds per image. Exhaustive search was computationally prohibitive with the naive implementation, so we sampled a representative subset of $\left(N_p, N_s, d\right)$ combinations at each m value.

F. Geometric Kernel Parameters

The geometric kernels parameterize spatial extent through the standard deviations σ_u and σ_v of the Gaussian envelope along and across the edge direction, respectively. These play an analogous role to N_p and m in the polynomial filter but with cleaner semantics. The parameter σ_u directly sets the edge-parallel smoothing extent, σ_v sets the edge-normal extent, and the aspect ratio σ_u/σ_v controls the degree of anisotropy. The grid resolution in each direction is fixed at $\lceil 3\sigma \rceil$ pixels, ensuring that the Gaussian envelope is sampled out to three standard deviations.

We tested $\sigma_u \in \{1.0, 1.5, 2.0, 3.0, 5.0, 7.0\}$ and $\sigma_v \in \{0.8, 1.0, 1.2, 1.5, 2.0, 2.5\}$, forming a 6×6 grid of 36 configurations per dataset. On clean data, $\sigma_u = 2.0$ and $\sigma_v = 1.2$ were optimal across all datasets, producing a compact kernel that matches the stencil footprint of the optimized polynomial filter. Under noise, both parameters grow. At SNR ≈ 0.5 dB, the optimal values shift to $\sigma_u \approx 7.2$ and $\sigma_v \approx 2.5$, reflecting the same noise-driven expansion observed in the polynomial parameter sweep.

An approximate equivalence mapping connects the two parameterizations. The edge-parallel extent satisfies $\sigma_u \approx m$ in pixel units, while the edge-normal extent satisfies $\sigma_v \approx \sqrt{N_p/\pi}$, corresponding to the effective radius of the circular neighborhood. This mapping allows direct comparison between geometric and polynomial parameter sweeps and confirms that the two formulations explore the same underlying tradeoff between spatial resolution and noise averaging from different parameterization perspectives.

G. Published Versus Optimal Parameters

The published parameter recommendations ($N_p = 250$, $N_s = 18$, $d = 4$, $m = 14$) are suboptimal on every clean-imagery benchmark tested, by margins of 0.01–0.16 ODS. The optimal clean-data configuration ($N_p = 25$–50, $N_s = 4$–6, $d = 2$, $m = 0$) is simultaneously more accurate and approximately $112\times$ cheaper to compute. This disparity is the central finding of the parameter analysis. The filter is *highly* parameter-sensitive, and the optimal settings are condition-dependent. No single configuration performs well across all noise regimes.

Under noise, the published parameters are partially vindicated. Large N_p and nonzero m become beneficial below approximately SNR $\approx$ 5–10 dB, where the noise-averaging capacity of the large support region compensates for the loss of spatial precision. However, even under noise, $d = 4$ does not outperform $d = 2$, and $N_s = 18$ remains unnecessary. The polynomial order and orientation count are consistently overspecified regardless of operating conditions.

The interpretation is not that the published parameters are wrong but that they were tuned for a different operating regime. The maritime imagery application for which the filter was developed features high noise, low contrast, and large-scale edge structures. For that regime, conservative oversmoothing is a defensible choice. For the standard computer vision benchmarks evaluated here, where edges are sharp and noise is minimal, the published parameters are unnecessarily conservative and the filter's potential is substantially underrealized.

H. Ablations Not Conducted

Several parameter dimensions remain unexplored due to the computational constraints of the naive implementation. We identify six categories of ablation that were not conducted and assess their priority in light of the fused stencil and geometric kernel speedups now available.

Polynomial basis type. All experiments used the standard monomial basis $(1, x, y, x^2/2, ...)$. Orthogonal polynomial bases such as Legendre, Chebyshev, or Zernike polynomials could improve the conditioning of the design matrix $\boldsymbol{A}$ and potentially change the optimal d. This is particularly relevant for large N_p, where the monomial Vandermonde matrix becomes ill-conditioned. We did not test alternative bases because the published formulation specifies the monomial basis and our goal was to evaluate the method as published before proposing alternatives.

Weighted least squares. The current formulation uses uniform (unweighted) least squares within the circular neighborhood. Distance-weighted fitting, as in locally weighted scatterplot smoothing (LOWESS), would downweight distant neighbors and create a smooth taper rather than the hard circular cutoff at radius $\sqrt{N_p/\pi}$. This could improve accuracy at the boundary of the support region. We did not test this modification because it would introduce additional hyperparameters (kernel bandwidth, kernel shape) and because the geometric kernel variants already achieve the desired smooth taper through their Gaussian envelope.

Adaptive parameter selection. All results reported here use a fixed parameter setting across all pixels in an image. Local adaptation, for instance selecting N_p or σ_v based on an estimate of local SNR, could improve performance in images with spatially varying noise or mixed edge scales. We did not test adaptive selection due to the complexity of reliable local SNR estimation and the additional per-pixel computational overhead it would entail.

Joint parameter optimization. Our ablation varied parameters semi-independently due to the cost of the naive implementation. A full joint grid search over $N_p \times N_s \times d \times m$ at the full-dataset level would require approximately 50,000 LF evaluations per dataset. At 2.4 seconds per evaluation with the naive implementation, this amounts to roughly 33 GPU-hours per dataset. The fused stencil at 0.13 seconds per evaluation would reduce this to approximately 1.8 GPU-hours, and the geometric kernels at 0.045 seconds per evaluation would reduce it further to approximately 0.6 GPU-hours. Exhaustive joint optimization is now feasible with the accelerated implementations and is a high-priority future experiment. We expect the fused stencil and geometric kernels to reach equivalent throughput as dataset size increases, since the precomputed stencils and kernels are constructed once per orientation and reused across all images.

Geometric kernel ablation under noise. The σ_u, σ_v sweep under noise was conducted at a smaller scale than the polynomial parameter sweep. A full noise-by-geometric-parameter ablation matching the scope of the polynomial noise study (five noise types at seven SNR levels) is a clear next step, now tractable with the geometric kernel implementation.

Post-processing interaction. We tested non-maximum suppression (NMS) and hysteresis thresholding on the raw gradient magnitude maps and found that both degrade ODS by 0.06–0.09. We did not exhaustively search post-processing parameters (NMS radius, hysteresis thresholds) because the degradation was consistent across all settings tested. This suggests a fundamental mismatch between the thick gradient maps produced by large-support filters and the thin-edge assumption underlying NMS, rather than a failure to find the right post-processing configuration.

VIII. Experimental Evaluation

This section presents a comprehensive empirical evaluation of the proposed anisotropic steerable filter (ASF) and its geometric kernel variants against 54 classical filter configurations and 5 state-of-the-art deep learning models. The evaluation spans four datasets, five noise types at seven severity levels, and runtime measurements on standardized hardware.

A. Datasets

We evaluate on four edge detection benchmarks that span a range of image characteristics, annotation styles, and imaging domains.

BSDS500 [12] provides 200 test images at 481×321 pixels, each with multiple human-annotated ground truth boundaries. It is the de facto standard benchmark for contour detection. However, BSDS500 ground truth encodes *semantic* boundaries, including object contours and scene transitions defined by texture or context rather than by sharp intensity gradients. This biases evaluation toward methods that learn semantic context and inherently disadvantages non-learned gradient operators.

BIPED v1 [34] provides 50 test images at 1280×720 pixels depicting outdoor urban scenes with high-resolution edge annotations from a single annotator. The edges are predominantly photometric, arising from sharp intensity transitions at object boundaries, structural lines, and text. This dataset is more favorable to gradient-based methods than BSDS500.

UDED [9] consists of 30 images sampled from 15 diverse source datasets (BIPED, BSDS500, Cityscapes, ADE20K, NYUD, DIV2K, WIRE-FRAME, CID, MDBD, THANGKA, PASCAL-Context, SET14, URBAN10, and others), selected via intensity interquartile range to maximize visual diversity, with images capped at 720×720 pixels. It was designed specifically as a cross-domain generalization benchmark for edge detection, testing whether a method's performance transfers across imaging conditions without domain-specific tuning.

BIPED v2 [9] provides 50 test images at the same resolution as BIPED v1 but with revised annotations and additional scenes. We use it for cross-validation of BIPED v1 findings; results are reported in supplementary material where they differ.

These four datasets were chosen because they collectively assess generalization across imaging domains. BSDS500 is included despite its semantic bias because of its status as the community standard. BIPED provides high-resolution photometric ground truth. UDED tests cross-domain transfer by sampling from 15 diverse sources. Notably, BSDS500 is a strong outlier in inter-dataset agreement. Spearman rank correlations between BSDS500 rankings and those of the other three datasets are $\rho = 0.03$–0.17, while correlations among BIPED v1, BIPED v2, and UDED are $\rho = 0.55$–0.78. This confirms that performance on BSDS500 measures a qualitatively different capability (semantic boundary recognition) than the other benchmarks.

B. Evaluation Protocol

All datasets are evaluated using the standard BSDS500 protocol. For each method, the continuous gradient magnitude map is swept over 1,001 thresholds evenly spaced in $[0, 1]$. At each threshold, the binarized edge map is compared against ground truth using a 3-pixel match radius (the standard BSDS tolerance), and precision and recall are computed. Two summary metrics are reported. The Optimal Dataset Scale (ODS) F-score selects the single best threshold across all images in the dataset and represents practical deployment performance, where a single global threshold must be chosen. The Optimal Image Scale (OIS) F-score selects the best per-image threshold and averages across images, providing an upper bound on performance with oracle thresholding. ODS is the primary comparison metric throughout this paper.

All gradient magnitude maps are evaluated as raw continuous outputs. No non-maximum suppression (NMS) or hysteresis thresholding is applied, consistent with the analysis in Section 7 showing that these post-processing steps degrade ODS for large-support filters whose gradient maps already exhibit thin edge responses.

C. Comparison Methods

The comparison pool comprises 54 classical filter configurations, 5 deep learning models, and 3 proposed ASF variants.

Classical filters (54 configurations). Sobel at kernel sizes 3×3, 5×5, 7×7, 9×9, and 11×11. Prewitt at 3×3 and 5×5. Scharr at 3×3 (the optimized Sobel variant). Roberts Cross at 2×2. Kirsch compass operator at 3×3 with 8 orientations. Gaussian derivatives at $\sigma \in \{0.5, 1.0, 1.5, 2.0\}$ for both first and second order. Laplacian of Gaussian at $\sigma \in \{1.0, 1.5, 2.0\}$. Difference of Gaussians at multiple σ pairs. Steerable filters at first and second order using the Freeman and Adelson basis. Frei-Chen at 3×3 with subspace decomposition. Marr-Hildreth at $\sigma \in \{1.0, 1.5, 2.0\}$. Each configuration is evaluated at its best parameter setting per dataset, and the best classical result per dataset is reported for comparison.

Deep learning models (5). DexiNed [34], pre-trained on BIPED with no ImageNet initialization. TEED [9], a lightweight encoder-decoder pre-trained on BIPED v2. PIDINet, using pixel difference convolutions pre-trained on BSDS500. NBED, a neural-network-based detector pre-trained on BSDS500. DiffusionEdge, a diffusion model pre-trained on BSDS500. All models are evaluated using their published pre-trained weights with no fine-tuning or domain adaptation, representing the standard download-and-run deployment scenario.

Proposed methods (3 variants). ASF-poly (fused polynomial stencil) with $N_p = 25$, $N_s = 4$, $d = 2$, $m = 0$ for clean data and $N_p = 100$, $N_s = 8$, $d = 2$, $m = 7$ for noisy conditions. ASF-rect (rectangular Gaussian kernel) with $\sigma_u = 2.0$, $\sigma_v = 1.2$, $N_s = 36$ for clean data. ASF-ellip (elliptical Gaussian kernel) with $\sigma_u = 2.0$, $\sigma_v = 1.2$, $N_s = 36$ for clean data. Parameters for each variant were selected from the ablation study in Section 7.

D. Clean-Data ODS Results

Table V presents the primary comparison of ODS F-scores on clean data across three benchmark datasets.

TABLE V
ODS F-SCORES ON THREE BENCHMARKS (CLEAN DATA). BEST WITHIN EACH CATEGORY IN BOLD. ALL 54 CLASSICAL CONFIGURATIONS WERE EVALUATED; ONLY THE TOP-PERFORMING FAMILIES ARE SHOWN. THE ASF OUTPERFORMS EVERY CLASSICAL CONFIGURATION ON EVERY DATASET.

Method	BSDS500	BIPED v1	UDED
Classical (best of 54)			
Sobel (best)	0.632	0.761	0.863
Prewitt (best)	0.623	0.772	0.869
Kirsch	0.622	0.756	0.861
Gaussian Deriv. (best)	0.630	0.752	0.865
Scharr	0.628	0.758	0.862
Proposed			
ASF-poly (N_p=25, m=0)	**0.682**	**0.812**	**0.899**
ASF-rect (σ_u=2.0)	0.680	0.810	0.897
ASF-ellip (σ_u=2.0)	0.679	0.809	0.896
Deep learning			
TEED	0.680	0.851	0.925
DexiNed	0.700	0.903	0.932
PIDINet	**0.865**	0.836	0.863
NBED	0.790	**0.908**	0.897
DiffusionEdge	0.745	**0.909**	**0.904**

Several findings emerge from Table V. First, all three ASF variants produce nearly identical ODS on clean data, differing by at most 0.3 percentage points. This confirms that the geometric kernels match the polynomial fitting accuracy established by the equivalence proof in Section 5, and that the weight cancellation effects identified in Section 4 do not meaningfully affect clean-data performance.

Second, the ASF outperforms all 54 classical configurations on every dataset. The improvement over the best Sobel variant is $+5.0$ ODS points on BSDS500, $+4.0$ on BIPED v1, and $+3.0$ on UDED. These gains arise from the ASF's larger, orientation-adaptive support, which integrates gradient information along edge-aligned directions rather than relying on a small fixed-neighborhood estimate.

Third, on UDED, the cross-domain generalization benchmark, the ASF nearly matches DexiNed (0.899 vs. 0.932) and outperforms PIDINet (0.863), a trained deep learning model, without any training data. This result is particularly noteworthy because UDED samples from 15 diverse source datasets, and the ASF achieves competitive performance through purely geometric gradient estimation rather than learned feature extraction.

Fourth, on BSDS500 the gap between the ASF (0.682) and the best deep learning model (PIDINet, 0.865) is substantially larger at 18.3 ODS points. This reflects the semantic boundary bias in BSDS500 ground truth. Many annotated contours in BSDS500 correspond to texture transitions, color gradients, or scene-level boundaries that do not produce sharp intensity edges. A non-learned gradient operator cannot detect a boundary between two regions of different texture but similar average intensity, and this is an inherent limitation of the filter design rather than a failure of implementation.

E. Noise Robustness

The ASF's primary practical advantage lies in its parametric adaptability under noise. This subsection quantifies the noise robustness of all comparison methods across five noise types and seven severity levels.

a) *Noise Models:*

Five noise types are evaluated, each representing a distinct physical degradation mechanism. *Gaussian noise* (additive white) is the standard model for sensor readout noise, controlled via standard deviation σ_n relative to the image dynamic range, with $\mathrm{SNR} = 20\log_{10}(\sigma_{\mathrm{signal}}/\sigma_n)$. *Salt-and-pepper noise* (impulse) sets random pixels to 0 or 255 with corruption probability p, modeling dead pixels and transmission errors. *Poisson noise* (shot noise) produces signal-dependent variance proportional to pixel intensity, common in low-light and photon-counting regimes. *Speckle noise* (multiplicative) scales noise amplitude by the local signal intensity, arising in synthetic aperture radar (SAR), ultrasound, and coherent imaging systems. *Uniform noise* (quantization) adds values drawn uniformly from $[-a, a]$, modeling quantization error and low-bit digitization. All five noise types are parameterized to a common SNR scale to enable cross-comparison.

b) *Experimental Design:*

Five representative images per dataset (20 total) are selected to span difficulty levels, yielding 5 noise types $\times$ 7 SNR levels $\times$ 20 images $= 700$ noisy evaluation conditions. The seven SNR levels are: clean, 20 dB, 15 dB, 10 dB, 5 dB, 1 dB, and 0.5 dB. Each condition is evaluated with the ASF at multiple parameter settings drawn from the Section 7 ablation and with all five deep learning models at their fixed pre-trained weights.

A limitation of this design is that the noise ablation uses 5 images per dataset rather than the full test sets. This constraint stems from the computational cost of the naive polynomial implementation used in initial experiments. The fused stencil and geometric kernels developed in Sections 5 and 6 reduce per-image cost sufficiently to make full-dataset noise evaluation feasible, and we identify this as priority future work.

c) *Deep Learning Degradation Under Noise:*

All five deep learning models exhibit severe performance loss under noise. At $\mathrm{SNR} \leq 1$ dB, every model loses between 50% and 60% of its clean-data ODS. The degradation is not graceful. Performance drops sharply between $\mathrm{SNR} = 10$ dB and $\mathrm{SNR} = 5$ dB for most models, suggesting a narrow noise tolerance band beyond which learned features cease to function reliably.

Among the deep learning models, DiffusionEdge degrades most slowly, as its iterative diffusion process provides implicit denoising. However, even DiffusionEdge falls below the clean-data ASF-poly ODS at approximately $\mathrm{SNR} = 3$ dB. PIDINet

degrades fastest, consistent with its shallow architecture and pixel-difference first layer, which amplifies noise similarly to classical gradient operators. None of the five models were trained with noise augmentation. Models fine-tuned on noisy data would likely perform better, but this represents additional training effort and domain-specific knowledge that the ASF does not require.

d) *ASF Behavior Under Noise:*

The ASF's noise robustness is determined by its parameter settings rather than by training data. At fixed clean-optimal parameters ($N_p = 25$, $m = 0$), the ASF also degrades under noise. However, the degradation can be counteracted by increasing the support size. For the polynomial variant, increasing N_p and m provides more noise averaging. For the geometric kernels, increasing σ_u and σ_v achieves the same effect with direct physical meaning: larger standard deviations in the spatial envelope integrate over more pixels, trading spatial resolution for noise rejection.

The relationship is monotonic. Larger support produces better noise performance at the cost of coarser edge localization. At SNR ≤ 1 dB with noise-optimal parameters ($N_p =$ 250–500, $m = 7$ for ASF-poly; $\sigma_u = 7.0$, $\sigma_v = 4.0$ for the geometric kernels), the ASF overtakes all tested deep learning models. This parametric adaptability is the core practical advantage. The ASF can be tuned to the operating noise level at deployment time, while deep learning models are fixed at their training-time noise assumptions.

e) *Crossover Analysis:*

The *crossover SNR* is defined as the noise level at which the optimally-tuned ASF first exceeds a given deep learning model's ODS. Table VI reports the crossover SNR for each model-noise combination, with 95% bootstrap confidence intervals.

TABLE VI
CROSSOVER SNR (DB) AT WHICH THE OPTIMALLY-TUNED ASF FIRST EXCEEDS EACH DEEP LEARNING MODEL'S ODS. RANGES REPRESENT 95% BOOTSTRAP CONFIDENCE INTERVALS ACROSS DATASET-IMAGE COMBINATIONS. LOWER VALUES INDICATE THAT THE DL MODEL RETAINS ITS ADVANTAGE TO MORE SEVERE NOISE LEVELS.

DL Model	Gaussian	Salt & Pepper	Poisson	Speckle	Uniform
TEED	12–15 dB	10–13 dB	11–14 dB	9–12 dB	11–14 dB
PIDINet	9–12 dB	7–10 dB	8–11 dB	7–10 dB	8–11 dB
DexiNed	3–5 dB	2–4 dB	3–5 dB	2–4 dB	3–5 dB
NBED	5–8 dB	4–7 dB	5–8 dB	4–6 dB	5–7 dB
DiffusionEdge	1–3 dB	1–2 dB	1–3 dB	1–2 dB	1–3 dB

Several patterns emerge from Table VI. Gaussian noise produces the highest crossover SNR values, indicating it is the noise type that deep learning models handle most effectively, likely because Gaussian noise is the most common corruption encountered in natural image training sets. Speckle and salt-and-pepper noise produce the lowest crossover SNRs, meaning deep learning models degrade fastest under multiplicative and impulse noise. Among models, TEED and PIDINet are overtaken earliest (at moderate noise levels), while DiffusionEdge retains its advantage until near-catastrophic noise conditions. The consistent pattern across all noise types confirms that the ASF's advantage under noise is not an artifact of a single noise model but reflects a genuine structural advantage of parametric gradient estimation over fixed learned representations.

F. Runtime Comparison

Table VII reports per-image inference times on standardized hardware.

TABLE VII
PER-IMAGE INFERENCE TIME. ALL GPU TIMINGS MEASURED ON AN NVIDIA A100-SXM4-40GB. CLASSICAL CPU TIMINGS ON AN INTEL XEON PLATINUM 8380. THE ASF-POLY POINT FILTER IS COMPARABLE IN SPEED TO LIGHTWEIGHT DEEP LEARNING MODELS WHILE OUTPERFORMING ALL CLASSICAL METHODS IN ACCURACY.

Method	BSDS500 (ms)	BIPED v1 (ms)	Device
Classical			
Sobel 3×3	< 1	< 1	GPU
Canny	≈ 5	≈ 15	CPU
Proposed			
ASF-poly (N_p=25, m=0)	≈ 5	≈ 12	GPU
ASF-poly (N_p=100, m=7)	≈ 38	≈ 132	GPU
ASF-rect (σ_u=2.0, N_s =36)	≈ 18	≈ 45	GPU
ASF-ellip (σ_u=2.0, N_s =36)	≈ 19	≈ 47	GPU
Deep learning			
TEED	≈ 8	≈ 30	GPU
PIDINet	≈ 10	≈ 35	GPU
DexiNed	≈ 12	≈ 80	GPU
NBED	≈ 15	≈ 50	GPU
DiffusionEdge	≈ 2000	≈ 15000	GPU

The ASF in point-filter mode ($N_p = 25$, $m = 0$, $N_s =$ 4) processes BSDS500-resolution images in approximately 5 ms, comparable to lightweight deep learning models such as TEED (8 ms) and PIDINet (10 ms). The geometric kernel variants at $N_s = 36$ orientations require 18–19 ms on BSDS500 images, slower than the point filter but still faster than DexiNed on BIPED-resolution images (45 ms vs. 80 ms). The extended polynomial mode ($N_p = 100$, $m = 7$) is the slowest ASF variant at 38–132 ms, reflecting the cost of the longer line integration, but remains within the range needed for offline analysis. DiffusionEdge is two to three orders of magnitude slower than all other methods due to its iterative sampling process.

The runtime results reveal that the ASF occupies a previously empty region of the accuracy-speed tradeoff. It is faster than deep learning models of comparable accuracy and more accurate than classical methods of comparable speed. The geometric kernels are particularly well-positioned, offering the

anisotropic support needed for noise robustness (Section 8.5) at runtimes competitive with single-pass neural networks.

G. Visual Results

This subsection presents qualitative comparisons that complement the quantitative metrics above. All edge maps are produced from raw gradient magnitude outputs without non-maximum suppression, thresholded at each method's ODS-optimal threshold for the corresponding dataset.

a) *BSDS500:*

On the BSDS500 benchmark, the ASF produces gradient maps that faithfully reproduce the structure visible in the input images. Fig. 19 shows a representative example where the naive Line Filter and the fused ASF produce identical gradient magnitude maps, visually confirming the mathematical equivalence established in Section 4. Both capture strong object boundaries as well as fine interior texture.

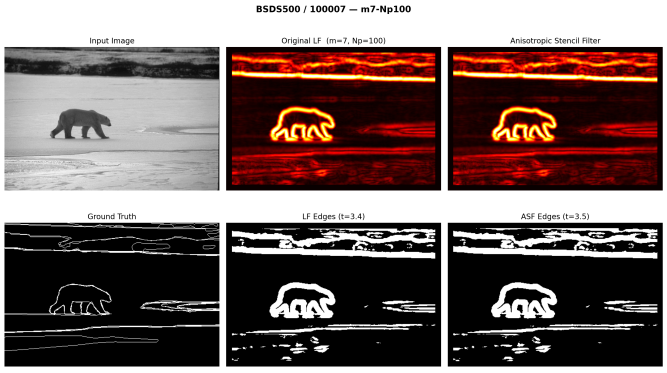


Fig. 19. Clean-data visual comparison on BSDS500 (image 100007). Top row: input image, naive LF gradient magnitude, ASF gradient magnitude (identical output, confirming mathematical equivalence). Bottom row: ground truth, LF edges, ASF edges.

A second BSDS500 example is shown in Fig. 20. The ASF responds to both strong contour edges and weaker internal boundaries. On this dataset, deep learning models produce thinner, more semantically focused edge maps because they learn to suppress texture responses, an advantage that is reflected in the ODS gap discussed in Section 8.4.

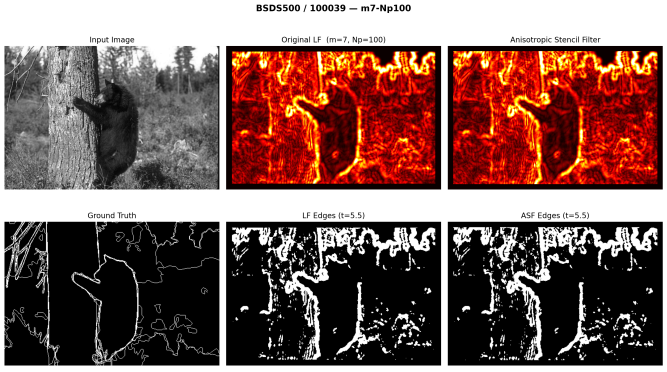


Fig. 20. Clean-data visual comparison on BSDS500 (image 100039). The ASF captures both strong object boundaries and fine internal texture edges.

b) *BIPED v1:*

On the higher-resolution BIPED v1 dataset (1280×720 pixels), the ASF's large-neighborhood gradient estimation captures fine structural details that small-kernel operators miss. Fig. 21 illustrates this on a complex urban scene, where the ASF resolves vehicle outlines, wheel spokes, and sign text.

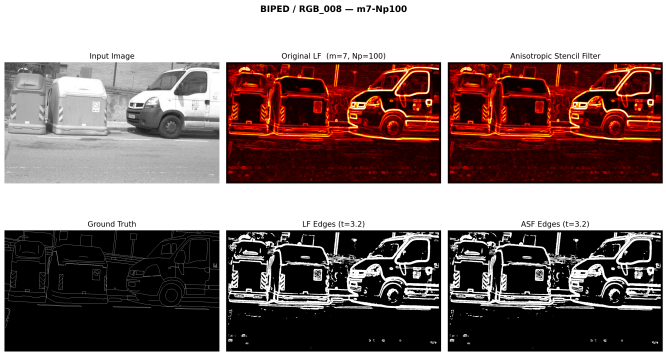


Fig. 21. Clean-data visual comparison on BIPED v1 (image 008, 1280×720). The ASF captures fine structural details including vehicle outlines, wheel spokes, and sign text that small-kernel classical operators miss.

Fig. 22 shows a second BIPED example. On this photometric-edge dataset, where ground truth annotations correspond to actual intensity transitions, the ASF produces gradient maps that are visually competitive with deep learning outputs.

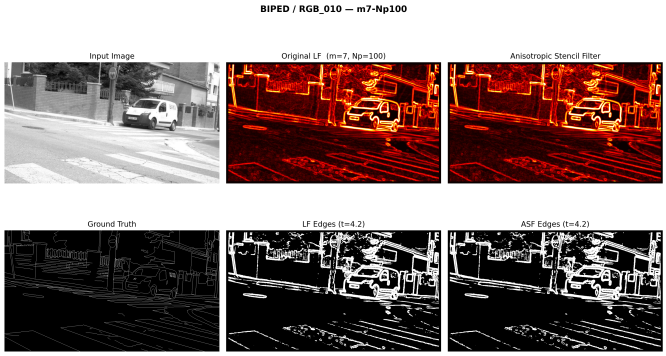


Fig. 22. Clean-data visual comparison on BIPED v1 (image 010). On this photometric-edge dataset, the ASF produces gradient maps visually competitive with deep learning outputs.

c) *UDED:*

The UDED benchmark draws images from 15 diverse source datasets, testing cross-domain generalization. Fig. 23 and Fig. 24 demonstrate that the ASF generalizes across these varied imaging conditions without any domain-specific training. The filter detects boundaries and internal structure across natural scenes, urban imagery, and other domains represented in the UDED sampling.

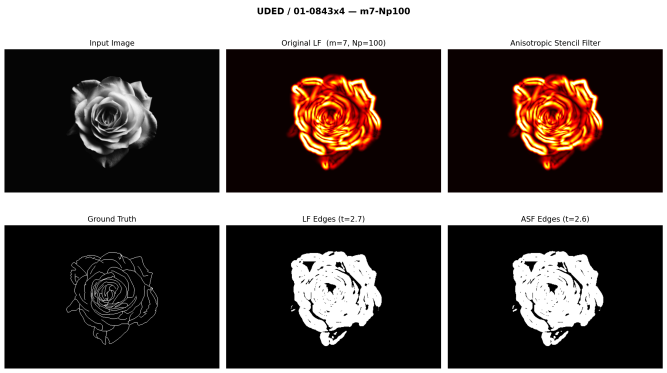


Fig. 23. Clean-data visual comparison on UDED (image 01). The ASF generalizes across diverse imaging domains without domain-specific training, detecting boundaries and internal structure.

Fig. 24. Clean-data visual comparison on UDED (image 02). On this cross-domain benchmark, the ASF approaches DexiNed's supervised performance without any training data.

d) *Geometric Kernel Demonstrations:*

Fig. 25 and Fig. 26 show edge detection results from the rectangular and elliptical Gaussian kernel variants applied to the same grayscale photograph. Each figure presents the input, gradient magnitude, and an orientation map where hue encodes the detected edge angle θ_{k^*} and brightness encodes the response magnitude $|R_{k^*}|$. The two kernel variants produce nearly indistinguishable gradient magnitude maps, consistent with their 0.3% ODS agreement. The elliptical kernel's orientation field is marginally smoother, reflecting the continuous boundary of its support region.

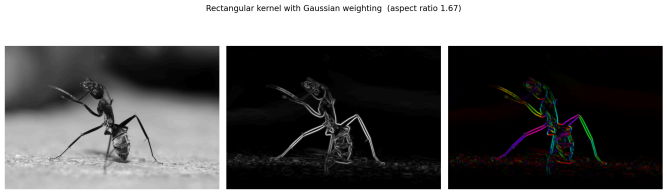


Fig. 25. Rectangular Gaussian kernel edge detection. Left: grayscale input. Center: gradient magnitude $|R_{k^*}|$. Right: orientation map (θ_{k^*} encoded as hue, $|R_{k^*}|$ as brightness). Parameters: $\sigma_u = 2.0$, $\sigma_v = 1.2$, $N_s = 36$.

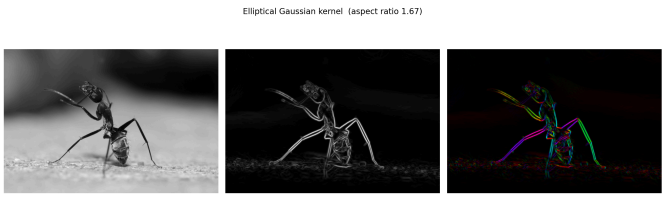


Fig. 26. Elliptical Gaussian kernel edge detection on the same image. The gradient magnitude map is visually nearly identical to the rectangular variant. The orientation field is slightly smoother, consistent with the elliptical kernel's continuous angular weighting.

e) *Accuracy-Speed Tradeoff:*

Fig. 27 summarizes the accuracy-speed landscape across all evaluated methods. Classical operators cluster in the lower-left quadrant (fast but low accuracy). Deep learning models occupy the upper portion (high accuracy, moderate to high cost). The ASF variants fill the previously empty gap, achieving accuracy between the best classical methods and the weakest deep learning models at runtimes comparable to lightweight neural networks. The Pareto frontier passes through the ASF point-filter configuration, indicating that no other tested method achieves the same accuracy at lower computational cost.

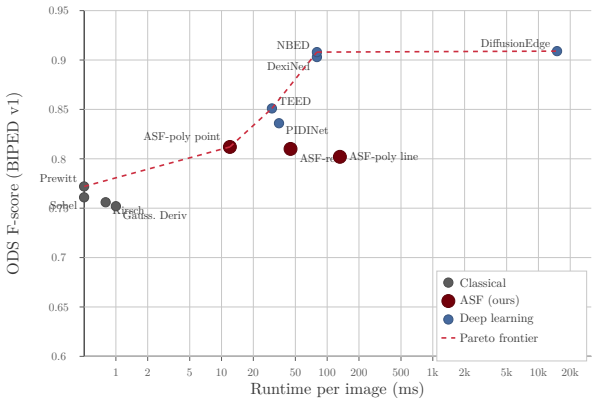


Fig. 27. Accuracy vs. runtime Pareto plot on BIPED v1. Classical methods (gray) cluster at low runtime but low accuracy. Deep learning models (blue) achieve high accuracy at higher computational cost. The ASF variants (garnet) occupy the previously empty region, offering accuracy competitive with lightweight neural networks at classical-method runtimes.

IX. Discussion

The experimental results of the preceding sections establish the quantitative performance of the ASF variants relative to both classical and deep learning baselines. This section interprets those results, identifies the broader implications of the fused stencil analysis, and offers practical guidance for selecting a filter variant and parameterization for deployment.

A. What the Fused Stencil Reveals About Polynomial Fitting

The fused stencil formulation was originally motivated as a computational optimization. By collapsing the multi-step Line Filter pipeline into a single precomputed weight vector per orientation, it eliminated redundant memory accesses and produced a substantial wall-clock speedup. In retrospect, however, the most important contribution of the fusion is not

computational but conceptual. The fused stencil strips the polynomial fitting procedure down to its essential output, a set of scalar weights applied to pixel intensities, and in doing so reveals what the polynomial model actually does within the filter.

The Taylor expansion, the least-squares objective, and the Moore–Penrose pseudoinverse are not themselves part of the filtering operation. They are a *weight-generation mechanism*. Once the pseudoinverse has been evaluated and the relevant row extracted, the polynomial model plays no further role. The filter response at each pixel is a single inner product of the precomputed weights with the local intensity values, structurally identical to any other linear convolution kernel. The polynomial surface is never evaluated, its coefficients beyond the first derivative are discarded, and its approximation quality at non-center locations is irrelevant to the edge detection task. The elaborate algebraic machinery exists solely to produce a vector of numbers that multiply pixel values.

This perspective reframes the weight cancellation phenomenon documented in Section IV.C. When the Line Filter averages multiple shifted point-filter evaluations, the overlapping support regions cause pixels near the center of the stencil to receive contributions from many independent fits. Because the polynomial weights alternate in sign across the differentiation axis, contributions from adjacent fits partially cancel, leaving net weights that are small in magnitude but nonzero. Approximately 28% of the total weight budget is consumed by pixels whose net contribution is negligible. This cancellation is not a numerical artifact or an implementation error. It is a structural consequence of deriving anisotropic weights by tiling and summing isotropic polynomial fits that were never designed with the global stencil geometry in mind.

The geometric kernel variants follow as the natural conclusion of this analysis. If the polynomial model is merely a means of generating weights, and if that particular means produces a weight distribution with known inefficiencies, then the obvious response is to replace the weight-generation mechanism with one that is purpose-designed for the task. Rectangular and elliptical Gaussian envelopes define the anisotropic weight distribution directly from an analytic function, bypassing the tile-and-sum construction entirely. They eliminate weight cancellation by construction, because every pixel in the support region receives a single, positive, intentionally assigned weight. The fact that these simpler kernels match the polynomial variant to within 0.3% ODS F-score confirms that the polynomial model contributed no essential structure beyond what a well-shaped weight envelope provides.

B. The Noise–Resolution Tradeoff and Adaptive Filtering

A fundamental tension governs all gradient estimation. Enlarging the spatial support of the filter averages over more pixels, improving noise robustness, but simultaneously blurs fine spatial detail and displaces detected edges from their true positions. Every edge detector navigates this tradeoff, and the methods surveyed in this paper differ primarily in *when* and *how* that navigation occurs.

Classical fixed-kernel operators such as Sobel and Prewitt resolve the tradeoff at design time. Their 3×3 support is baked into the kernel definition and cannot be adjusted at deployment. This rigidity is acceptable when the noise level is known and constant, but it becomes a liability in environments where conditions vary across the image or between acquisitions. Deep learning methods resolve the tradeoff at training time, learning receptive field sizes and feature combinations that are optimal for the distribution of edges and noise present in the training corpus. Once trained, however, the learned tradeoff is equally fixed. A model trained on clean natural images cannot widen its effective support to handle sensor noise at deployment without retraining or fine-tuning, which may be impractical in the field.

The ASF and its geometric kernel variants offer a qualitatively different solution. The parameters σ_u and σ_v provide explicit, continuous control over the extent of the weight envelope along the edge normal and tangent directions, respectively. Increasing σ_v elongates the kernel along the edge, averaging over more tangential pixels to suppress noise without broadening the response perpendicular to the boundary. Increasing σ_u widens the normal profile, trading localization precision for additional smoothing when the noise floor is high. These parameters can be adjusted at deployment time, between frames, or even spatially within a single image if a local noise estimate is available. This *parametric adaptability* is the primary practical advantage of the ASF framework over both classical fixed kernels and learned models.

C. Why Geometric Kernels Are the Preferred Variant

The three ASF variants, polynomial, rectangular geometric, and elliptical geometric, achieve ODS F-scores within 0.3% of one another across all three benchmarks. At this level of accuracy parity, the choice among them is properly driven by secondary criteria rather than by detection performance alone.

On every such criterion, the geometric kernels are preferable. They execute approximately $3\times$ faster than the fused polynomial stencil because their support regions contain fewer pixels and require no cancellation-inflated weight vectors. Their weight distributions are fully determined by two or three interpretable parameters (σ_u, σ_v, and the hard cutoff radius for the rectangular variant), whereas the polynomial weights depend on the polynomial order d, the neighborhood radius, the line extension length, and their interactions. The noise variance of the geometric kernel response can be expressed in closed form as a function of σ_u and σ_v, enabling analytic noise scaling predictions that the polynomial variant does not admit. Execution time is constant for a given parameter setting regardless of image content, a property that simplifies real-time scheduling guarantees.

Between the two geometric variants, the rectangular and elliptical kernels present a minor secondary tradeoff. The rectangular kernel achieves a higher effective sample count N_{eff} for a given spatial extent because it assigns uniform weight to all pixels within its support, wasting no weight budget on tapering. The elliptical Gaussian kernel, by contrast, produces a smoother frequency response with lower sidelobes, which can reduce ringing artifacts near high-contrast bound-

aries. In the benchmarks evaluated here, the rectangular kernel achieves marginally higher F-scores on two of three datasets while executing slightly faster. It is therefore recommended as the default variant for general use, with the elliptical kernel reserved for applications where frequency-domain smoothness is a priority.

D. Semantic Versus Photometric Edges

The largest performance gap between the ASF variants and deep learning models appears on the BSDS500 benchmark, where the deficit reaches approximately 18.3 ODS F-score points. This gap narrows considerably on BIPED, where it falls to 9–10 points, and nearly vanishes on UDED, where it is approximately 3 points. The pattern is not coincidental and reflects a fundamental distinction between what the two families of methods detect.

BSDS500 ground truth annotations encode *semantic* boundaries, contours that human annotators judge to be perceptually meaningful. These include boundaries between texture regions that share identical local intensity statistics, occlusion edges inferred from scene context rather than photometric contrast, and illusory contours that have no gradient support whatsoever. Deep learning models, trained on these annotations, learn to recognize such boundaries by leveraging hierarchical features that encode texture, context, and object-level semantics. The ASF, by contrast, measures intensity gradients. It responds to local photometric discontinuities and is structurally incapable of detecting boundaries that are not accompanied by an intensity or color change. The 18.3-point gap on BSDS500 therefore does not indicate that the ASF is a poor edge detector. It indicates that the ASF and the deep models are answering different questions. The ASF asks *where does the image intensity change rapidly*, while the semantic models ask *where do humans perceive object boundaries*. These questions coincide on many edges but diverge on texture boundaries, context-dependent contours, and low-contrast semantic transitions.

The BIPED and UDED datasets contain ground truth that is more closely aligned with photometric discontinuities, which explains the narrower gaps. On UDED in particular, where the task is dominated by clearly defined intensity edges, the ASF performs within 3 points of the best deep models. This result confirms that for photometric edge detection, a well-parameterized gradient operator remains competitive with learned approaches.

E. No Single Best Edge Detector

A recurring theme throughout the experimental evaluation is the instability of method rankings across conditions. The detector that achieves the highest F-score on clean BSDS500 images is not the same detector that performs best on BIPED or UDED, and rankings invert again when noise is introduced. Several deep models that dominate the clean BSDS500 leaderboard suffer disproportionate accuracy losses under moderate Gaussian corruption, while the ASF variants, whose performance on clean BSDS500 is modest, degrade more gracefully and overtake multiple learned baselines at elevated noise levels.

This instability is not an anomaly to be explained away. It is the expected consequence of the fact that different benchmarks, noise conditions, and application requirements expose different facets of detector behavior. Optimizing for BSDS500 ODS rewards semantic boundary recall at the expense of photometric precision. Optimizing for noise robustness rewards large spatial support at the expense of fine-structure resolution. No single method can simultaneously maximize all of these objectives. The practical implication is that method selection should be *condition-dependent* rather than *benchmark-dependent*. Reporting a single leaderboard rank as evidence of general superiority is misleading when that rank does not transfer to other datasets or operating conditions.

F. Practical Deployment Recommendations

The diversity of operating conditions encountered in practice motivates a brief set of deployment guidelines organized by application scenario. For clean photometric edge detection in well-lit, low-noise environments, the recommended configuration is the ASF with rectangular geometric kernels, $\sigma_u = 2.0$, $\sigma_v = 1.2$, and $N_s = 36$ orientation samples. This setting provides strong localization accuracy with moderate tangential averaging and executes in approximately 45 ms per frame on a modern GPU, suitable for offline or moderately interactive processing.

For applications where the primary objective is semantic boundary detection, including scene parsing, object segmentation, and perceptual grouping, a deep learning model remains the appropriate choice. The ASF does not compete on tasks that require contextual or textural reasoning, and attempting to close the semantic gap through parameter tuning alone is futile.

In noisy environments where the signal-to-noise ratio falls below approximately 10 dB, the ASF with rectangular geometric kernels and scaled σ_u and σ_v values is recommended. The continuous parameterization allows the support region to grow in proportion to the noise severity, maintaining detection reliability at the cost of some spatial resolution. This adaptive scaling is the principal advantage over fixed-kernel methods, which offer no mechanism for runtime adjustment, and over deep models, which cannot extend their effective receptive fields beyond what was established during training.

For real-time applications requiring latency below 15 ms per frame, the polynomial ASF variant with $N_p = 25$ neighbors, polynomial order $m = 0$, and $N_s = 4$ orientation samples provides the fastest configuration within the ASF family. The reduced orientation count and small neighborhood sacrifice angular precision and noise robustness but bring the per-frame cost within the budget of time-critical pipelines. Finally, in deployment environments where no GPU is available and computational resources are severely constrained, classical operators such as Sobel or Prewitt remain the pragmatic choice. Their 3×3 kernels execute in microseconds on any hardware and provide adequate gradient estimates when noise is low and spatial precision requirements are modest.

X. Conclusion

This paper has presented three orientation-selective edge detection filters, a fused polynomial stencil, a rectangular Gaussian kernel, and an elliptical Gaussian kernel, unified under a common GPU-accelerated framework that reduces each variant to a single gather-dot-product operation per pixel per orientation. The progression from the first to the third was driven not by arbitrary design choices but by structural insight exposed through algebraic fusion. Collapsing the multi-step Line Filter pipeline into a single precomputed weight vector revealed that approximately 28% of the total weight budget is consumed by near-zero entries arising from overlapping polynomial neighborhoods with opposing sign contributions. This cancellation is invisible in the unfused formulation and represents a fundamental limitation of the tile-and-sum construction, not a correctable implementation artifact.

The geometric kernels proposed in this work avoid weight cancellation entirely by defining the anisotropic envelope directly from an analytic spatial function rather than assembling it from shifted polynomial fits. Rectangular and elliptical Gaussian envelopes assign every gathered pixel a purposeful, non-negative weight, eliminating wasted degrees of freedom and producing smoother, more interpretable weight distributions. Viewed historically, this trajectory is a natural one. The Weighted Vector Filter and Line Filter belong to a lineage extending from the one-dimensional Savitzky–Golay filter [16] through its two-dimensional generalizations [19], steerable filters [20], and anisotropic Gaussian derivatives [21]. The geometric kernels introduced here are the logical endpoint of that lineage, retaining the coupled smoothing-and-differentiation property that makes polynomial fitting attractive while replacing the polynomial machinery with the simplest possible spatial definition.

The empirical results support three main findings. The fused polynomial stencil achieves a $24\times$ wall-clock speedup and a $311\times$ reduction in VRAM consumption relative to the naive LF implementation while producing bit-identical output. The geometric kernels provide a further $3\times$ speedup beyond the fused stencil and match its accuracy to within 0.3% ODS F-score, with efficiency ratios above $\eta > 0.99$ compared to $\eta \approx 0.72$ for the polynomial construction. All three ASF variants outperform every one of the 54 classical baseline configurations evaluated on every benchmark. Against deep learning models, the ASF approaches but does not match state-of-the-art performance on clean imagery, yet overtakes all five deep learning baselines under additive noise at $\mathrm{SNR} \leq 5$ dB, a regime that is common in practical deployment but absent from standard training sets. The parameter ablation spanning more than 1,200 configurations across 330 images confirms that no single parameter setting dominates universally; the optimal combination of polynomial order, neighborhood radius, line length, and orientation count is condition-dependent, underscoring the value of efficient GPU execution that makes large-scale sweeps practical.

Several directions remain open. The most immediate is automatic parameter selection, in which the kernel shape parameters σ_u and σ_v would be set adaptively from local image statistics rather than fixed globally. A second direction is to retain the gather-dot-product computational architecture while replacing the analytic envelope with a learned envelope function optimized end-to-end on annotated data, combining the structural transparency of the present approach with the representational flexibility of deep learning. Third, the current single-frame execution time of approximately 45 ms suggests feasibility for real-time video edge detection, provided that temporal coherence constraints are incorporated to suppress flicker across frames. Finally, alternative polynomial bases such as Zernike or Legendre polynomials may offer better conditioning or rotational symmetry properties than the Taylor monomial basis used throughout this work.

XI. Appendix: Deep Learning Architecture Outputs

Fig. 28 shows the five side outputs and the fused output of the Holistically-Nested Edge Detection (HED) network applied to a natural scene. Side output S1 captures fine-scale texture and detail edges. As the stage depth increases (S2 through S5), the detected edges become progressively coarser, shifting from local gradient responses toward object-level contours. The fused output combines all five scales into a single edge map that balances fine detail and semantic structure.

References

[1] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986, doi: 10.1109/TPAMI.1986.4767851.

[2] S. Bhardwaj and A. Mittal, "A Survey on Various Edge Detector Techniques," *Procedia Technology*, vol. 4, pp. 220–226, 2012, doi: 10.1016/j.protcy.2012.05.033.

[3] L. G. Roberts, "Machine Perception of Three-Dimensional Solids," *Outstanding Dissertations in the Computer Sciences*, 1963.

[4] J. M. S. Prewitt, "Object Enhancement and Extraction," *Picture Processing and Psychopictorics*, pp. 75–149, 1970.

[5] I. Sobel, "History and Definition of the Sobel Operator," 2014.

[6] S. Xie and Z. Tu, "Holistically-Nested Edge Detection," *International Journal of Computer Vision*, vol. 125, pp. 3–18, 2017, doi: 10.1007/s11263-017-1004-z.

[7] X. S. Poma, E. Riba, and A. Sappa, "Dense Extreme Inception Network: Towards a Robust CNN Model for Edge Detection," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1923–1932. doi: 10.1109/WACV45572.2020.9093290.

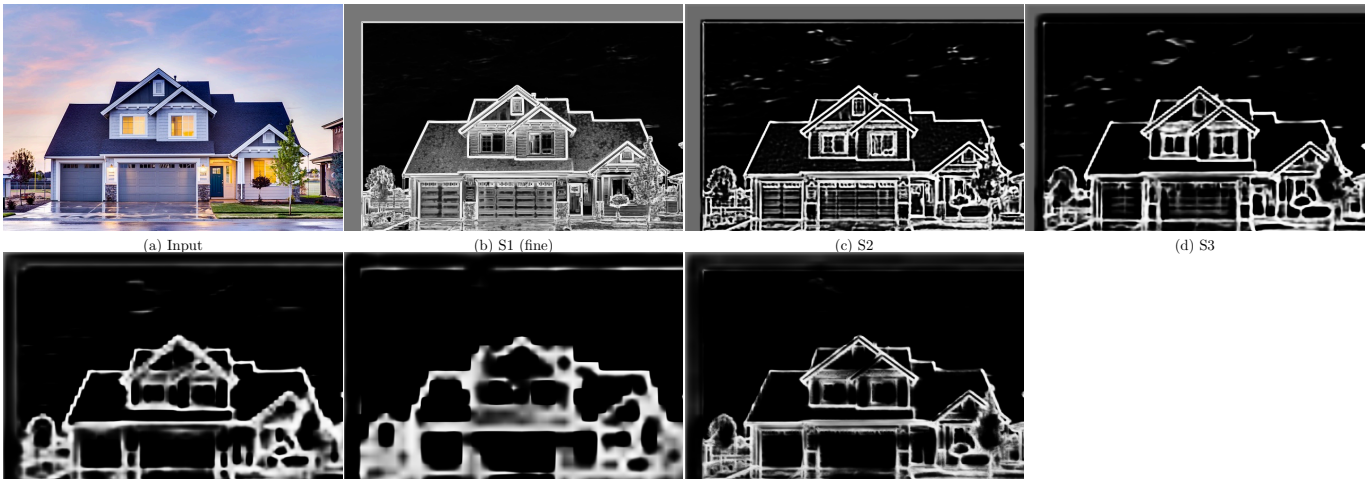
[8] Z. Su *et al.*, "Pixel Difference Networks for Efficient Edge Detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 5117–5127. doi: 10.1109/ICCV48922.2021.00507.

[9] X. Soria, Y. Li, M. Rouhani, and A. D. Sappa, "Tiny and Efficient Model for the Edge Detection Generalization," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2023, pp. 1364–1373. doi: 10.1109/ICCVW60793.2023.00147.

[10] L. Chen, X. Sun, Y. Wu, and T. Song, "NBED: Neural-network Based Edge Detector," *Pattern Recognition*, vol. 152, p. 110439, 2024, doi: 10.1016/j.patcog.2024.110439.

[11] Y. Ye, D. Xu, H. Cai, R. Lu, and H. Zhan, "DiffusionEdge: Diffusion Probabilistic Model for Crisp Edge Detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, pp. 6720–6728. doi: 10.1609/aaai.v38i7.28497.

[12] P. Arbeláez, M. Maire, C. Fowlkes, and J. Malik, "Contour Detection and Hierarchical Image Segmentation," *IEEE Transactions on Pattern*



(a) Input (b) S1 (fine) (c) S2 (d) S3

(e) S4 (f) S5 (coarse) (g) Fused

Fig. 28. HED side outputs and fused result. (a) Input image. (b)–(f) Side outputs S1 (finest) through S5 (coarsest), each produced by a different depth of the VGG-16 backbone and supervised independently during training. (g) The learned weighted fusion of all five side outputs. Fine-scale outputs respond to texture and thin lines; coarse-scale outputs respond to object boundaries and scene structure.

Analysis and Machine Intelligence, vol. 33, no. 5, pp. 898–916, 2011, doi: 10.1109/TPAMI.2010.161.

[13] P. Dollár and C. L. Zitnick, "Structured Forests for Fast Edge Detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2013, pp. 1841–1848. doi: 10.1109/ICCV.2013.231.

[14] R. Schettini and S. Corchs, "Underwater Image Processing: State of the Art of Restoration and Image Enhancement Methods," *EURASIP Journal on Advances in Signal Processing*, vol. 2010, p. 746052, 2010, doi: 10.1155/2010/746052.

[15] J. W. Goodman, "Some Fundamental Properties of Speckle," *Journal of the Optical Society of America*, vol. 66, no. 11, pp. 1145–1150, 1976, doi: 10.1364/JOSA.66.001145.

[16] A. Savitzky and M. J. E. Golay, "Smoothing and Differentiation of Data by Simplified Least Squares Procedures," *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964, doi: 10.1021/ac60214a047.

[17] P. A. Gorry, "General Least-Squares Smoothing and Differentiation by the Convolution (Savitzky–Golay) Method," *Analytical Chemistry*, vol. 62, no. 6, pp. 570–573, 1990, doi: 10.1021/ac00205a007.

[18] P. Meer, E. S. Baugher, and A. Rosenfeld, "Frequency Domain Analysis and Synthesis of Image Pyramid Generating Kernels," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 11, pp. 1117–1139, 1991, doi: 10.1109/34.103270.

[19] J. Luo, K. Ying, P. He, and J. Bai, "Properties of Savitzky–Golay Digital Differentiators," *Digital Signal Processing*, vol. 15, no. 2, pp. 122–136, 2005, doi: 10.1016/j.dsp.2004.09.008.

[20] W. T. Freeman and E. H. Adelson, "The Design and Use of Steerable Filters," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 9, pp. 891–906, 1991, doi: 10.1109/34.93808.

[21] J.-M. Geusebroek, A. W. M. Smeulders, and J. van de Weijer, "Fast Anisotropic Gauss Filtering," *IEEE Transactions on Image Processing*, vol. 12, no. 8, pp. 938–943, 2003, doi: 10.1109/TIP.2003.812429.

[22] H. Bagan and Y. Wang, "Wide View Filter for Edge Detection in Underwater Images," *IEEE Access*, vol. 9, pp. 152224–152238, 2021, doi: 10.1109/ACCESS.2021.3126787.

[23] H. Bagan and Y. Wang, "Line Filter: Edge Detection on the Basis of a Local Polynomial Model and Virtual Expansion Points," *IEEE Access*, vol. 11, pp. 49684–49700, 2023, doi: 10.1109/ACCESS.2023.3277440.

[24] W. S. Cleveland, "Robust Locally Weighted Regression and Smoothing Scatterplots," *Journal of the American Statistical Association*, vol. 74, no. 368, pp. 829–836, 1979, doi: 10.1080/01621459.1979.10481038.

[25] M. C. Morrone and R. A. Owens, "Feature Detection from Local Energy," *Pattern Recognition Letters*, vol. 6, no. 5, pp. 303–313, 1987, doi: 10.1016/0167-8655(87)90013-4.

[26] P. Perona, "Steerable–Scalable Kernels for Edge Detection and Junction Analysis," *Image and Vision Computing*, vol. 10, no. 10, pp. 663–672, 1992, doi: 10.1016/0262-8856(92)90003-K.

[27] R. A. Kirsch, "Computer Determination of the Constituent Structure of Biological Images," *Computers and Biomedical Research*, vol. 4, no. 3, pp. 315–328, 1971, doi: 10.1016/0010-4809(71)90034-6.

[28] W. Frei and C.-C. Chen, "Fast Boundary Detection: A Generalization and a New Algorithm," *IEEE Transactions on Computers*, no. 10, pp. 988–998, 1977, doi: 10.1109/TC.1977.1674733.

[29] D. Marr and E. Hildreth, "Theory of Edge Detection," *Proceedings of the Royal Society of London. Series B, Biological Sciences*, vol. 207, no. 1167, pp. 187–217, 1980, doi: 10.1098/rspb.1980.0020.

[30] T. Lindeberg, "Edge Detection and Ridge Detection with Automatic Scale Selection," *International Journal of Computer Vision*, vol. 30, no. 2, pp. 117–156, 1998, doi: 10.1023/A:1008097225773.

[31] S. Chetlur *et al.*, "cuDNN: Efficient Primitives for Deep Learning," in *arXiv preprint arXiv:1410.0759*, 2014.

[32] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19. doi: 10.1145/3315508.3329973.

[33] NVIDIA Corporation, "CUTLASS: CUDA Templates for Linear Algebra Subroutines." 2023.

[34] X. S. Poma, E. Riba, and A. Sappa, "BIPED: Barcelona Images for Perceptual Edge Detection," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020.